

app.py — Function Flowchart

Dr Neal Aggarwal · drnealaggarwal.info · Flask web application · 1908 lines · *Click any box to jump to source*

App Init Helper Data / Constants Route Engine Function Misc

APPLICATION BOOTSTRAP

Flask App
app = Flask(__name__)
Creates Flask instance. secret_key from env or dev fallback.

TAG_URLS
dict[str, str] — ~60 entries
Maps technology tags (Claude API, PyTorch, FHIR...) to Wikipedia/docs URLs. Used by Skills page.

Path Constants
BASE / POSTS_DIR / DATA_DIR
BASE = app.py directory. POSTS_DIR = content/posts/. DATA_DIR = data/.

HELPER FUNCTIONS

parse_frontmatter()
text: str → (dict, str)
Splits Markdown into YAML frontmatter and body. Parses key: value pairs from --- block.

load_posts()
limit: int = None → list[dict]
Globs content/posts/*.md. Parses frontmatter + generates excerpt (first non-HTML/image para, ≤300 chars). Returns slug, title, date, category, tags, level, read_time, summary, featured.

load_post()
slug: str → dict | None
Reads single post. Renders Markdown → HTML via python-markdown (fenced_code, tables, toc, attr_list). Returns full post dict with rendered content.

load_json()
filename: str → dict | list
Reads + JSON-parses from data/. Returns {} if not found. Used for skills.json, library.json, demos.json.

FLASK ROUTES

index()
GET /
Loads all posts. Separates featured (first 1) from recent (next 8). → index.html

archive()
GET /archive
Loads all posts in reverse-chronological order. → archive.html

post()
GET /post/<slug>
Loads post by slug (404 if missing). Finds prev/next by position in full post list. → post.html

skills()
GET /skills
Loads skills.json + TAG_URLS for tag linkification. → skills.html

library()
GET /library
Loads library.json. Applies ?cat= and ?author= query-string filters. → library.html

demos()
GET /demos
Loads demos.json. → demos.html

backtester()
GET /backtester
Renders backtester UI. Page POSTs AJAX to /backtester/run.

backtester_run()
POST /backtester/run
Receives JSON config → calls _run_backtest() → returns equity curve, drawdown, B&H curve, metrics.

nlp_extractor()
GET /nlp-extractor
Renders clinical NLP extractor UI.

nlp_extractor_run()
POST /nlp-extractor/run
Receives {{text}} → calls _run_nlp_extractor() → returns {{entities, counts, total}}.

voice_notes()
GET /voice-notes
Renders dictation / voice note UI.

voice_notes_format()
POST /voice-notes/format
Receives {{transcript}} → calls _format_clinical_note() → returns structured plain-text note.

drug_checker()
GET /drug-checker
Renders drug interaction checker UI.

drug_checker_run()
POST /drug-checker/run
Receives {{text}} → NLP extraction + line-by-line parse → calls _check_drug_interactions() → returns interactions JSON.

agent_trace()
GET /agent-trace
Renders LLM agent trace visualiser UI.

agent_trace_parse()

POST /agent-trace/parse

Receives {{trace}} → calls _parse_agent_trace() → returns {{steps, model, token counts, tool_counts}}.

binaural()

GET /binaural

Renders Web Audio API binaural beat generator.

about()

GET /about

Static about page.

contact()

GET + POST /contact

GET: renders form. POST: validates fields, sends via Gmail SMTP SSL port 465 (GMAIL_USER + GMAIL_APP_PASS env vars). Flash + redirect.

**BACKEND ENGINE FUNCTIONS****_run_backtest(config) → dict**

Pure-Python ML backtesting. Synthetic OHLCV (numpy geometric random walk). Features: SMA, EMA, RSI(14), momentum, volatility, MACD. Fits LogisticRegression / RandomForest / GradientBoosting (scikit-learn). Long-only simulation. Returns equity + B&H curves, drawdown, dates, Sharpe, max_dd, win_rate, n_trades, accuracy.

_format_clinical_note(transcript) → str

Dictation-to-note formatter. Strips filler words, sentence-splits, runs NLP extractor, classifies sentences into PRESENTING COMPLAINT / HISTORY / EXAMINATION / INVESTIGATIONS / ASSESSMENT / PLAN sections. Appends structured medication + diagnosis lists with datetime header.

_parse_agent_trace(raw) → dict

Normalises Anthropic API, OpenAI Chat Completion, or generic messages-array traces. Handles text, thinking, tool_use, tool_result content blocks. OpenAI tool_calls + role='tool'. Token counts + tool_counts. Returns {{steps, model, total_tokens, tool_counts}}.

_run_nlp_extractor(text) → dict

Rule-based clinical NLP. ~200 medications, ~200 diagnoses. Detects 6 entity types: DATES (8 patterns), DOSAGES (7), VITAL_SIGNS (11), LAB_VALUES (20), MEDICATIONS, DIAGNOSES. Non-overlapping span list. Returns {{entities, counts, total}}.

_check_drug_interactions(drug_list) → dictNormalises names via ~100 aliases (tylenol → paracetamol, eliquis → apixaban...). Checks all $O(n^2)$ pairs against 60+ interaction records (CONTRAINDICATED / MAJOR / MODERATE). Returns interactions sorted by severity with mechanism, effect, recommendation, onset.**Misc Handlers**

service_worker() — GET /sw.js with correct MIME type for PWA. not_found() — 404 handler → 404.html. __main__ — dev server on 0.0.0.0:5001, debug=True.

LARGE STATIC DATA STRUCTURES**_NLP_MEDICATIONS**

list[str] — ~200 names

Cardiovascular, antibiotics, analgesics, psychiatric, respiratory, diabetes, GI, thyroid, rheumatology, haematology. Used by _run_nlp_extractor and _check_drug_interactions.

_NLP_DIAGNOSES

list[str] — ~200 names

Cardiovascular, respiratory, metabolic, renal, GI, musculoskeletal, neurological, psychiatric, infections, oncology/haematology. Used by _run_nlp_extractor.

_DRUG_ALIASES

dict[str, str] — ~100 entries

Brand/alternate → generic:
tylenol → paracetamol, eliquis → apixaban,
coumadin → warfarin,
glucophage → metformin, etc.**_DRUG_INTERACTIONS**

list[dict] — 60+ records

Each: drugs pair, severity, mechanism, effect, recommendation, onset.
Covers: MAOI+SSRI, PDE5i+nitrates, warfarin+enzyme modulators, digoxin+amiodarone, opioid+benzo, QTc prolongers, triple whammy.

app.py — Source (1908 lines) · Dr Neal Aggarwal · drnealaggarwal.info

```

"""
1  Dr Neal Aggarwal — AI Engineering Site
  Flask application. All content lives in data/ (JSON) and content/posts/ (Markdown).
2  To add a post: drop a .md file in content/posts/ with YAML frontmatter.
  To update the library or skills: edit data/library.json or data/skills.json.
3  """
4  import os
  import json
5  import re
  from pathlib import Path
6  from datetime import datetime
7
  import smtplib
  from email.mime.text import MIMEText
8  from email.mime.multipart import MIMEMultipart
9
  import markdown
  from flask import Flask, render_template, abort, request, redirect, url_for, flash, jsonify
10
  app = Flask(__name__)
11 app.secret_key = os.environ.get("SECRET_KEY", "dev-secret-neal-site-2026")
12
  TAG_URLS = {
    "Claude API": "https://www.anthropic.com/api",
13    "OpenAI": "https://en.wikipedia.org/wiki/OpenAI",
    "Gemini": "https://en.wikipedia.org/wiki/Gemini_(language_model)",
14    "RAG": "https://en.wikipedia.org/wiki/Retrieval-augmented_generation",
    "tool use": "https://en.wikipedia.org/wiki/Prompt_engineering",
15    "prompt engineering": "https://en.wikipedia.org/wiki/Prompt_engineering",
    "Claude Code": "https://www.anthropic.com/claude-code",
16    "multi-agent": "https://en.wikipedia.org/wiki/Multi-agent_system",
    "MCP": "https://modelcontextprotocol.io/introduction",
17    "autonomous execution": "https://en.wikipedia.org/wiki/Autonomous_agent",
    "agent SDKs": "https://en.wikipedia.org/wiki/Software_development_kit",
18    "tool servers": "https://modelcontextprotocol.io/docs/concepts/tools",
    "connectors": "https://modelcontextprotocol.io/docs/concepts/resources",
19    "local + remote": "https://modelcontextprotocol.io/docs/concepts/transport",
    "PyTorch": "https://en.wikipedia.org/wiki/PyTorch",
20    "TensorFlow": "https://en.wikipedia.org/wiki/TensorFlow",
    "transformers": "https://en.wikipedia.org/wiki/Transformer_(deep_learning_architecture)",
21    "LoRA": "https://en.wikipedia.org/wiki/Fine-tuning_(deep_learning)",
    "PEFT": "https://en.wikipedia.org/wiki/Fine-tuning_(deep_learning)",
22    "HuggingFace": "https://en.wikipedia.org/wiki/Hugging_Face",
    "scikit-learn": "https://en.wikipedia.org/wiki/Scikit-learn",
23    "XGBoost": "https://en.wikipedia.org/wiki/XGBoost",
    "statsmodels": "https://www.statsmodels.org",
24    "scipy": "https://en.wikipedia.org/wiki/SciPy",
    "R": "https://en.wikipedia.org/wiki/R_(programming_language)",
25    "spaCy": "https://en.wikipedia.org/wiki/SpaCy",
    "NLTK": "https://en.wikipedia.org/wiki/Natural_Language_Toolkit",
26    "BERT": "https://en.wikipedia.org/wiki/BERT_(language_model)",
    "clinical NLP": "https://en.wikipedia.org/wiki/Natural_language_processing",
27    "embeddings": "https://en.wikipedia.org/wiki/Word_embedding",
    "vector search": "https://en.wikipedia.org/wiki/Nearest_neighbor_search",
28    "diagnostic AI": "https://en.wikipedia.org/wiki/Artificial_intelligence_in_healthcare",
    "risk models": "https://en.wikipedia.org/wiki/Risk_assessment",
29    "FHIR": "https://en.wikipedia.org/wiki/Fast_Healthcare_Interoperability_Resources",
    "HL7": "https://en.wikipedia.org/wiki/Health_Level_7",
30    "DICOM": "https://en.wikipedia.org/wiki/DICOM",
    "pandas": "https://en.wikipedia.org/wiki/Pandas_(software)",
31    "data pipelines": "https://en.wikipedia.org/wiki/Pipeline_(computing)",
    "anonymisation": "https://en.wikipedia.org/wiki/Data_anonymization",
32    "Arduino": "https://en.wikipedia.org/wiki/Arduino",
    "ESP32": "https://en.wikipedia.org/wiki/ESP32",
33    "ESP-IDF": "https://docs.espressif.com/projects/esp-idf/en/latest/",
    "Raspberry Pi": "https://en.wikipedia.org/wiki/Raspberry_Pi",
34    "LoRa": "https://en.wikipedia.org/wiki/LoRa",
    "I2C/SPI": "https://en.wikipedia.org/wiki/I%C2%B2C",
35    "Python": "https://en.wikipedia.org/wiki/Python_(programming_language)",
    "Flask": "https://en.wikipedia.org/wiki/Flask_(web_framework)",
36    "FastAPI": "https://en.wikipedia.org/wiki/FastAPI",
    "C/C++": "https://en.wikipedia.org/wiki/C%2B%2B",
37    "LISP": "https://en.wikipedia.org/wiki/Lisp_(programming_language)",
    "EMACS": "https://en.wikipedia.org/wiki/Emacs",
38    "MQL5": "https://www.mql5.com/en/articles",
    "MetaTrader": "https://en.wikipedia.org/wiki/MetaTrader_4",
  }

```

```

39     "quant finance":      "https://en.wikipedia.org/wiki/Quantitative_analysis_(finance)",
40     "neural nets":        "https://en.wikipedia.org/wiki/Artificial_neural_network",
41     "backtesting":        "https://en.wikipedia.org/wiki/Backtesting",
42     "Bitcoin":            "https://en.wikipedia.org/wiki/Bitcoin",
43     "Ethereum":           "https://en.wikipedia.org/wiki/Ethereum",
44     "Solidity":           "https://en.wikipedia.org/wiki/Solidity",
45     "Cardano":            "https://en.wikipedia.org/wiki/Cardano_(blockchain_platform)",
46     "cryptography":       "https://en.wikipedia.org/wiki/Cryptography",
47 }
48
49 BASE = Path(__file__).parent
50 POSTS_DIR = BASE / "content" / "posts"
51 DATA_DIR = BASE / "data"
52
53 # — helpers —
54
55 def parse_frontmatter(text: str) -> tuple[dict, str]:
56     """Extract YAML-style frontmatter from a markdown file."""
57     meta: dict = {}
58     body = text
59     if text.startswith("---"):
60         end = text.find("\n---", 3)
61         if end != -1:
62             fm_block = text[3:end].strip()
63             body = text[end + 4:].strip()
64             for line in fm_block.splitlines():
65                 if ":" in line:
66                     k, _, v = line.partition(":")
67                     meta[k.strip()] = v.strip().strip('\"').strip('\'')
68     return meta, body
69
70 def load_posts(limit: int = None) -> list[dict]:
71     posts = []
72     for path in sorted(POSTS_DIR.glob("*.md"), reverse=True):
73         raw = path.read_text(encoding="utf-8")
74         meta, body = parse_frontmatter(raw)
75         # Generate excerpt — first non-empty paragraph
76         excerpt_src = re.split(r'\n\n', body.strip(), maxsplit=3)
77         excerpt = ""
78         for chunk in excerpt_src:
79             stripped = chunk.strip().lstrip("#").strip()
80             if stripped and not stripped.startswith("![") and not stripped.startswith("<"):
81                 excerpt = stripped[:300]
82                 if len(stripped) > 300:
83                     excerpt += "..."
84             break
85     posts.append({
86         "slug": path.stem,
87         "title": meta.get("title", path.stem.replace("-", " ").title()),
88         "date": meta.get("date", ""),
89         "category": meta.get("category", "AI"),
90         "tags": [t.strip() for t in meta.get("tags", "").split(",") if t.strip()],
91         "level": meta.get("level", ""),
92         "read_time": meta.get("read_time", ""),
93         "summary": meta.get("summary", excerpt),
94         "featured": meta.get("featured", "false").lower() == "true",
95     })
96     if limit:
97         return posts[:limit]
98     return posts
99
100 def load_post(slug: str) -> dict | None:
101     path = POSTS_DIR / f"{slug}.md"
102     if not path.exists():
103         return None
104     raw = path.read_text(encoding="utf-8")
105     meta, body = parse_frontmatter(raw)
106     html = markdown.markdown(
107         body,
108         extensions=["fenced_code", "tables", "toc", "attr_list"],
109     )
110     return {
111         "slug": slug,
112         "title": meta.get("title", slug.replace("-", " ").title()),
113         "date": meta.get("date", ""),
114         "category": meta.get("category", "AI"),

```

```

79     "tags": [t.strip() for t in meta.get("tags", "").split(",") if t.strip()],
80     "level": meta.get("level", ""),
81     "read_time": meta.get("read_time", ""),
82     "summary": meta.get("summary", ""),
83     "content": html,
84 }
85
86 def load_json(filename: str) -> dict | list:
87     path = DATA_DIR / filename
88     if path.exists():
89         return json.loads(path.read_text(encoding="utf-8"))
90     return {}
91
92 # — routes —————
93
94 @app.route("/")
95 def index():
96     posts = load_posts()
97     featured = [p for p in posts if p["featured"]][:1]
98     recent = [p for p in posts if not p["featured"][:8]
99     return render_template("index.html", posts=posts, featured=featured, recent=recent)
100
101 @app.route("/archive")
102 def archive():
103     posts = load_posts()
104     return render_template("archive.html", posts=posts)
105
106 @app.route("/post/<slug>")
107 def post(slug):
108     p = load_post(slug)
109     if not p:
110         abort(404)
111     all_posts = load_posts()
112     idx = next((i for i, x in enumerate(all_posts) if x["slug"] == slug), None)
113     prev_post = all_posts[idx + 1] if idx is not None and idx + 1 < len(all_posts) else None
114     next_post = all_posts[idx - 1] if idx is not None and idx > 0 else None
115     return render_template("post.html", post=p, prev_post=prev_post, next_post=next_post)
116
117 @app.route("/skills")
118 def skills():
119     data = load_json("skills.json")
120     return render_template("skills.html", skills=data, tag_urls=TAG_URLS)
121
122 @app.route("/library")
123 def library():
124     data = load_json("library.json")
125     category = request.args.get("cat", "all")
126     author = request.args.get("author", "all")
127     items = data.get("items", [])
128     if category != "all":
129         items = [i for i in items if i.get("type") == category]
130     if author != "all":
131         items = [i for i in items if i.get("author", "").lower() == author.lower()]
132     authors = sorted(set(i.get("author", "") for i in data.get("items", [])))
133     return render_template("library.html", items=items, authors=authors,
134                           current_cat=category, current_author=author)
135
136 @app.route("/demos")
137 def demos():
138     data = load_json("demos.json")
139     return render_template("demos.html", demos=data.get("demos", []))
140
141 # — Backtester —————
142
143 def _run_backtest(config: dict) -> dict:
144     """Pure-Python ML backtesting engine. No external data required."""
145     import numpy as np
146     import pandas as pd
147     from sklearn.linear_model import LogisticRegression
148     from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
149     from sklearn.preprocessing import StandardScaler

```

```

11 seed = int(config.get("seed", 42))
12 n_days = int(config.get("n_days", 500))
11 capital0 = float(config.get("capital", 10000))
3 model_type = config.get("model", "logistic")
11 features = config.get("features", ["sma", "rsi", "momentum"])
11 train_pct = float(config.get("train_pct", 0.7))
4
# — Synthetic OHLCV (geometric random walk with slight upward drift) —
11 rng = np.random.default_rng(seed)
5 daily_ret = rng.normal(0.0005, 0.015, n_days)
11 closes = pd.Series(100.0 * np.cumprod(1 + daily_ret))
6
# — Features —————
feat: dict = {}
11
if "sma" in features:
7     feat["sma_ratio"] = closes.rolling(10).mean() / closes.rolling(30).mean() - 1
11
if "ema" in features:
8     feat["ema_ratio"] = closes.ewm(span=12).mean() / closes.ewm(span=26).mean() - 1
11
if "rsi" in features:
9     delta = closes.diff()
12     gain = delta.clip(lower=0).rolling(14).mean()
0     loss = (-delta).clip(lower=0).rolling(14).mean()
12     feat["rsi"] = 100 - 100 / (1 + gain / (loss + 1e-9))
1
if "momentum" in features:
12     feat["mom_5"] = closes.pct_change(5)
2     feat["mom_20"] = closes.pct_change(20)
12
if "volatility" in features:
12     r = closes.pct_change()
3     feat["vol_10"] = r.rolling(10).std()
12     feat["vol_ratio"] = r.rolling(10).std() / (r.rolling(30).std() + 1e-9)
4
if "macd" in features:
12     ema12 = closes.ewm(span=12).mean()
12     ema26 = closes.ewm(span=26).mean()
5     macd = ema12 - ema26
12     sig9 = macd.ewm(span=9).mean()
6     feat["macd_hist"] = (macd - sig9) / (closes + 1e-9)
12
X_all = pd.DataFrame(feat)
12 y_all = (closes.shift(-1) > closes).astype(int)
7
combined = X_all.copy()
12 combined["target"] = y_all
8 combined["price"] = closes
12 combined.dropna(inplace=True)
9
feat_cols = list(feat.keys())
12 X_mat = combined[feat_cols].values
13 y_vec = combined["target"].values
0 prices_vec = combined["price"].values
13
split = int(train_pct * len(X_mat))
1 X_tr, X_te = X_mat[:split], X_mat[split:]
13 y_tr, y_te = y_vec[:split], y_vec[split:]
2 p_te = prices_vec[split:]
13
scaler = StandardScaler()
13 X_tr_sc = scaler.fit_transform(X_tr)
3 X_te_sc = scaler.transform(X_te)
13
# — Model —————
4 if model_type == "rf":
13     model = RandomForestClassifier(n_estimators=100, max_depth=5, random_state=seed)
5 elif model_type == "gb":
13     model = GradientBoostingClassifier(n_estimators=100, max_depth=3, random_state=seed)
6 else:
13     model = LogisticRegression(max_iter=1000)
13
model.fit(X_tr_sc, y_tr)
13 signals = model.predict(X_te_sc)
7 accuracy = float((signals == y_te).mean())
13
# — Backtest (long-only, fully invested when signal=1) —————

```

```

8     capital = capital0
    position = 0.0
13    entry_px = 0.0
9     equity = [capital]
    bnh = [capital]
14    trades = []
0     bnh_shares = capital / p_te[0]

    for i in range(len(p_te) - 1):
1     px = p_te[i]
        sig = signals[i]

14        if sig == 1 and position == 0.0:
2            position = capital / px
14            entry_px = px
3            capital = 0.0
14        elif sig == 0 and position > 0.0:
4            capital = position * px
            trades.append((px - entry_px) / entry_px)
14            position = 0.0

5            mtm = capital if position == 0.0 else position * p_te[i + 1]
            equity.append(mtm)
14            bnh.append(bnh_shares * p_te[i + 1])
6

    if position > 0.0:
14        trades.append((p_te[-1] - entry_px) / entry_px)
7

    eq = np.array(equity, dtype=float)
14    bnh = np.array(bnh, dtype=float)
8

    # — Metrics —
14    ret_arr = np.diff(eq) / (eq[:-1] + 1e-9)
9    sharpe = float(np.sqrt(252) * ret_arr.mean() / (ret_arr.std() + 1e-9))
    tot_ret = float((eq[-1] - eq[0]) / eq[0])
15    bnh_ret = float((bnh[-1] - bnh[0]) / bnh[0])
0    run_max = np.maximum.accumulate(eq)
    dd = (eq - run_max) / (run_max + 1e-9)
15    max_dd = float(dd.min())
1    n_tr = len(trades)
    win_rate = float(sum(1 for p in trades if p > 0) / max(n_tr, 1))

15    dates = pd.bdate_range(end="2026-06-19", periods=len(eq)).strftime("%Y-%m-%d").tolist()

15    return {
3        "equity": [round(v, 2) for v in eq.tolist()],
        "bnh": [round(v, 2) for v in bnh.tolist()],
15        "drawdown": [round(v * 100, 3) for v in dd.tolist()],
4        "dates": dates,
        "metrics": {
15            "total_return": round(tot_ret * 100, 2),
5            "bnh_return": round(bnh_ret * 100, 2),
            "sharpe": round(sharpe, 3),
15            "max_drawdown": round(max_dd * 100, 2),
6            "win_rate": round(win_rate * 100, 1),
            "n_trades": n_tr,
15            "accuracy": round(accuracy * 100, 1),
7        },
    }
15
8
@app.route("/backtester")
15 def backtester():
9     return render_template("backtester.html")
16
0 @app.route("/backtester/run", methods=["POST"])
15 def backtester_run():
16     try:
1            config = request.get_json(force=True) or {}
            result = _run_backtest(config)
16            return jsonify({"ok": True, **result})
2     except Exception as exc:
            return jsonify({"ok": False, "error": str(exc)}), 500
16
3
# — Clinical Text NLP Extractor —
16
4 _NLP_MEDICATIONS = [

```

```

- # Cardiovascular
16 "metoprolol", "bisoprolol", "atenolol", "carvedilol", "nebivolol",
5 "lisinopril", "ramipril", "enalapril", "perindopril", "captopril", "trandolapril",
- "amlodipine", "nifedipine", "diltiazem", "verapamil", "felodipine",
16 "atorvastatin", "simvastatin", "rosuvastatin", "pravastatin", "fluvastatin",
6 "warfarin", "apixaban", "rivaroxaban", "dabigatran", "edoxaban",
- "aspirin", "clopidogrel", "ticagrelor", "prasugrel", "dipyridamole",
16 "furosemide", "spironolactone", "bumetanide", "torasemide", "eplerenone",
7 "digoxin", "amiodarone", "flecainide", "sotalol", "dronedarone",
- "nitroglycerine", "isosorbide", "hydralazine", "doxazosin", "prazosin",
16 "sacubitril", "valsartan", "ivabradine", "empagliflozin", "dapagliflozin",
8 # Antibiotics
- "amoxicillin", "ampicillin", "co-amoxiclav", "flucloxacillin",
16 "penicillin", "piperacillin", "tazobactam", "piperacillin-tazobactam",
9 "ciprofloxacin", "levofloxacin", "moxifloxacin", "ofloxacin",
- "metronidazole", "tinidazole",
17 "doxycycline", "tetracycline", "minocycline",
0 "azithromycin", "clarithromycin", "erythromycin",
- "trimethoprim", "nitrofurantoin", "fosfomycin", "co-trimoxazole",
17 "ceftriaxone", "cefuroxime", "cephalexin", "cefazolin", "ceftazidime",
1 "vancomycin", "teicoplanin", "linezolid", "daptomycin", "tigecycline",
- "meropenem", "ertapenem", "imipenem", "doripenem",
17 "gentamicin", "amikacin", "tobramycin",
2 # Analgesics / Anti-inflammatory
- "paracetamol", "acetaminophen",
17 "ibuprofen", "naproxen", "diclofenac", "celecoxib", "meloxicam", "indomethacin", "etoricoxib",
3 "morphine", "oxycodone", "hydromorphone", "fentanyl", "alfentanil", "remifentanyl",
- "codeine", "dihydrocodeine", "tramadol", "buprenorphine", "tapentadol",
17 "prednisolone", "dexamethasone", "hydrocortisone", "methylprednisolone", "betamethasone",
4 # Psychiatric / Neurological
- "sertraline", "fluoxetine", "paroxetine", "citalopram", "escitalopram", "fluvoxamine",
17 "venlafaxine", "duloxetine", "desvenlafaxine", "mirtazapine", "trazodone", "agomelatine",
5 "amitriptyline", "nortriptyline", "clomipramine", "imipramine", "dosulepin",
- "diazepam", "lorazepam", "clonazepam", "alprazolam", "temazepam", "nitrazepam", "oxazepam",
17 "quetiapine", "olanzapine", "risperidone", "aripiprazole", "haloperidol", "clozapine", "amisulpride",
6 "lithium", "valproate", "lamotrigine", "carbamazepine", "levetiracetam", "phenytoin", "topiramate",
- "donepezil", "rivastigmine", "galantamine", "memantine",
17 "levodopa", "pramipexole", "ropinirole", "rasagiline", "selegiline", "entacapone",
7 "sumatriptan", "rizatriptan", "zolmitriptan", "propranolol",
- "pregabalin", "gabapentin",
17 "melatonin", "zopiclone", "zolpidem",
8 # Respiratory
- "salbutamol", "terbutaline", "salmeterol", "formoterol", "indacaterol", "olodaterol",
17 "ipratropium", "tiotropium", "umeclidinium", "glycopyrronium", "aclidinium",
9 "beclometasone", "fluticasone", "budesonide", "ciclesonide", "mometasone",
- "montelukast", "zafirlukast", "theophylline", "aminophylline",
18 "acetylcysteine", "carbocysteine", "dornase alfa",
0 "roflumilast", "nintedanib", "pirfenidone",
- # Diabetes
18 "metformin", "gliclazide", "glibenclamide", "glimepiride", "glipizide",
1 "sitagliptin", "saxagliptin", "linagliptin", "alogliptin", "vildagliptin",
- "canagliflozin", "ertugliflozin",
18 "liraglutide", "semaglutide", "exenatide", "dulaglutide", "lixisenatide",
2 "insulin", "glargine", "detemir", "degludec", "aspart", "lispro", "glulisine", "isophane",
- "pioglitazone", "acarbose", "repaglinide",
18 # Gastrointestinal
3 "omeprazole", "lansoprazole", "pantoprazole", "esomeprazole", "rabeprazole",
- "ranitidine", "famotidine", "cimetidine",
18 "ondansetron", "metoclopramide", "domperidone", "prochlorperazine", "cyclizine", "granisetron",
4 "loperamide", "codeine phosphate",
- "lactulose", "movicol", "senna", "bisacodyl", "docusate", "macrogol",
18 "mesalazine", "sulfasalazine", "infliximab", "adalimumab", "vedolizumab", "ustekinumab",
5 "cholestyramine", "colesevelam",
- # Thyroid / Endocrine
18 "levothyroxine", "liothyronine", "carbimazole", "propylthiouracil",
6 "alendronate", "risedronate", "denosumab", "strontium", "teriparatide",
- "anastrozole", "letrozole", "exemestane", "tamoxifen", "fulvestrant",
18 "testosterone", "estradiol", "progesterone", "norethisterone", "etonogestrel",
7 "finasteride", "dutasteride",
- # Renal / Rheumatology
18 "allopurinol", "febuxostat", "colchicine", "benzbromarone",
8 "hydroxychloroquine", "methotrexate", "azathioprine", "mycophenolate", "leflunomide",
- "ciclosporin", "tacrolimus", "sirolimus",
18 "rituximab", "tocilizumab", "etanercept", "abatacept", "certolizumab", "golimumab",
9 "belimumab", "secukinumab", "ixekizumab", "baricitinib", "tofacitinib",
- # Anticoagulation / Haematology
19 "heparin", "enoxaparin", "dalteparin", "fondaparinux", "tinzaparin",
0 "erythropoietin", "epoetin", "darbepoetin",
- "ferrous sulphate", "ferrous fumarate", "iron sucrose", "ferric carboxymaltose",

```

```

19 "folic acid","hydroxocobalamin","cyanocobalamin",
1 "filgrastim","lenograstim","pegfilgrastim",
- # Miscellaneous
19 "calcium carbonate","calcium acetate","sevelamer","lanthanum",
2 "vitamin d","cholecalciferol","alfacalcidol","calcitriol",
- "bisoprolol","carvedilol",
19 "sildenafil","tadalafil","vardenafil",
3 ]

19 NLP_DIAGNOSES = [
4 # Cardiovascular
- "hypertension","essential hypertension","secondary hypertension","resistant hypertension",
19 "heart failure","congestive heart failure","cardiac failure","systolic heart failure",
5 "diastolic heart failure","heart failure with reduced ejection fraction",
- "atrial fibrillation","atrial flutter","supraventricular tachycardia","ventricular tachycardia",
19 "ventricular fibrillation","complete heart block","sick sinus syndrome",
6 "myocardial infarction","ST elevation myocardial infarction","STEMI","NSTEMI",
- "angina","stable angina","unstable angina","Prinzmetal angina",
19 "acute coronary syndrome","ACS",
7 "stroke","ischaemic stroke","haemorrhagic stroke","subarachnoid haemorrhage",
- "transient ischaemic attack","TIA",
19 "deep vein thrombosis","DVT","pulmonary embolism","PE",
8 "peripheral vascular disease","peripheral arterial disease",
- "aortic stenosis","mitral regurgitation","aortic regurgitation","mitral stenosis",
19 "cardiomyopathy","dilated cardiomyopathy","hypertrophic cardiomyopathy","restrictive cardiomyopathy",
9 "pericarditis","endocarditis","myocarditis","cardiac tamponade",
- # Respiratory
20 "asthma","brittle asthma","exercise-induced asthma",
0 "chronic obstructive pulmonary disease","COPD","emphysema","chronic bronchitis",
- "pneumonia","community-acquired pneumonia","hospital-acquired pneumonia","aspiration pneumonia",
20 "pulmonary fibrosis","idiopathic pulmonary fibrosis","IPF",
1 "pleural effusion","pneumothorax","haemothorax","haemopneumothorax",
- "pulmonary oedema","cardiogenic pulmonary oedema","acute respiratory distress syndrome","ARDS",
20 "pulmonary hypertension","cor pulmonale",
2 "sleep apnoea","obstructive sleep apnoea","OSA",
- "bronchiectasis","cystic fibrosis",
20 "COVID-19","SARS-CoV-2","influenza",
3 "sarcoidosis","hypersensitivity pneumonitis","mesothelioma",
- # Metabolic / Endocrine
20 "type 2 diabetes","type 1 diabetes","diabetes mellitus","gestational diabetes",
4 "hypothyroidism","hyperthyroidism","thyrotoxicosis","thyroid storm",
- "Grave's disease","Hashimoto's thyroiditis","autoimmune thyroiditis",
20 "hypercholesterolaemia","dyslipidaemia","hyperlipidaemia","hypertriglyceridaemia",
5 "obesity","morbid obesity","metabolic syndrome",
- "Cushing's syndrome","Addison's disease","primary adrenal insufficiency",
20 "hyperparathyroidism","hypoparathyroidism",
6 "hypokalaemia","hyperkalaemia","hyponatraemia","hypernatraemia",
- "hypomagnesaemia","hypocalcaemia","hypercalcaemia",
20 "acromegaly","hypopituitarism","diabetes insipidus","SIADH",
7 # Renal
- "chronic kidney disease","CKD","acute kidney injury","AKI","acute tubular necrosis",
20 "nephrotic syndrome","nephritic syndrome","proteinuria",
8 "glomerulonephritis","IgA nephropathy","membranous nephropathy",
- "polycystic kidney disease","renal artery stenosis","renal cell carcinoma",
20 "urinary tract infection","UTI","cystitis","pyelonephritis","prostatitis",
9 # Gastrointestinal
- "peptic ulcer disease","gastric ulcer","duodenal ulcer","H pylori",
21 "Crohn's disease","ulcerative colitis","inflammatory bowel disease","IBD",
0 "irritable bowel syndrome","IBS",
- "gastro-oesophageal reflux","GORD","GERD","Barrett's oesophagus","oesophagitis",
21 "cirrhosis","alcoholic liver disease","non-alcoholic fatty liver disease","NAFLD","NASH",
1 "pancreatitis","acute pancreatitis","chronic pancreatitis",
- "cholecystitis","cholelithiasis","gallstones","cholangitis","primary sclerosing cholangitis",
21 "diverticulitis","diverticular disease","diverticulosis",
2 "appendicitis","bowel obstruction","volvulus","intussusception",
- "coeliac disease","malabsorption","short bowel syndrome",
21 "hepatitis","hepatitis B","hepatitis C","autoimmune hepatitis",
3 # Musculoskeletal
- "rheumatoid arthritis","osteoarthritis","psoriatic arthritis","reactive arthritis",
21 "gout","pseudogout","calcium pyrophosphate deposition",
4 "osteoporosis","osteopenia","osteomalacia","Paget's disease",
- "systemic lupus erythematosus","SLE","lupus nephritis",
21 "ankylosing spondylitis","fibromyalgia","polymyalgia rheumatica",
5 "Sjogren's syndrome","systemic sclerosis","scleroderma",
- "polymyositis","dermatomyositis",
21 # Neurological
6 "epilepsy","seizure disorder","generalised epilepsy","focal epilepsy","status epilepticus",
- "Parkinson's disease","Parkinsonism","Lewy body dementia",
21 "Alzheimer's disease","dementia","vascular dementia","frontotemporal dementia",

```

```

7     "multiple sclerosis", "MS", "relapsing remitting MS", "primary progressive MS",
-     "migraine", "cluster headache", "tension headache", "trigeminal neuralgia",
21    "peripheral neuropathy", "diabetic neuropathy", "Charcot-Marie-Tooth",
8     "Guillain-Barré syndrome", "myasthenia gravis", "motor neurone disease", "ALS",
-     "Bell's palsy", "carpal tunnel syndrome", "sciatica", "spinal stenosis",
21    "meningitis", "encephalitis", "brain abscess", "subdural haematoma", "extradural haematoma",
9     "normal pressure hydrocephalus", "Huntington's disease", "cerebellar ataxia",
-     # Psychiatric
22    "depression", "major depressive disorder", "treatment-resistant depression",
0     "bipolar disorder", "bipolar affective disorder", "mania", "hypomania",
-     "anxiety", "generalised anxiety disorder", "GAD", "panic disorder", "social anxiety disorder",
22    "schizophrenia", "schizoaffective disorder", "psychosis", "first episode psychosis",
1     "PTSD", "post-traumatic stress disorder",
-     "obsessive-compulsive disorder", "OCD", "body dysmorphic disorder",
22    "eating disorder", "anorexia nervosa", "bulimia nervosa", "ARFID",
2     "ADHD", "attention deficit hyperactivity disorder", "autism spectrum disorder",
-     "alcohol use disorder", "substance misuse", "opiate dependence",
22    # Infections
3     "sepsis", "septic shock", "bacteraemia", "SIRS",
-     "cellulitis", "erysipelas", "necrotising fasciitis", "abscess",
22    "tuberculosis", "TB", "latent TB", "multi-drug resistant TB",
4     "HIV", "AIDS", "opportunistic infection",
-     "Lyme disease", "brucellosis", "malaria", "typhoid",
22    "COVID-19", "SARS-CoV-2", "influenza", "RSV",
5     "hepatitis B", "hepatitis C",
-     "infective endocarditis",
22    # Cancer / Haematology
6     "breast cancer", "lung cancer", "colorectal cancer", "colon cancer", "rectal cancer",
-     "prostate cancer", "ovarian cancer", "cervical cancer", "endometrial cancer",
22    "lymphoma", "Hodgkin lymphoma", "non-Hodgkin lymphoma", "diffuse large B-cell lymphoma",
7     "leukaemia", "chronic lymphocytic leukaemia", "CLL", "acute myeloid leukaemia", "AML",
-     "acute lymphoblastic leukaemia", "ALL", "chronic myeloid leukaemia", "CML",
22    "multiple myeloma", "myelodysplastic syndrome", "MDS",
8     "melanoma", "basal cell carcinoma", "squamous cell carcinoma",
-     "hepatocellular carcinoma", "cholangiocarcinoma", "pancreatic cancer",
22    "bladder cancer", "kidney cancer", "thyroid cancer", "gastric cancer",
9     "brain tumour", "glioblastoma", "glioma", "meningioma",
-     "anaemia", "iron deficiency anaemia", "pernicious anaemia", "aplastic anaemia",
23    "thrombocytopenia", "thrombophilia", "antiphospholipid syndrome",
0     # Other
-     "chronic fatigue syndrome", "fibromyalgia", "sarcoidosis",
23    "psoriasis", "eczema", "dermatitis", "urticaria", "angioedema",
1     "glaucoma", "macular degeneration", "diabetic retinopathy", "cataracts",
-     "hypothyroidism", "hyperthyroidism",
23    ]
2
23 def _run_nlp_extractor(text: str) -> dict:
3     """Rule-based clinical NLP entity extractor using regex + curated medical vocabulary."""
-     entities: list[dict] = []
23    seen_spans: list[tuple] = []
4
-     def no_overlap(s: int, e: int) -> bool:
23        return all(e <= a or s >= b for a, b in seen_spans)
5
-     # — DATES —————
23    date_patterns = [
6        r'\b\d{1,2}[/\-\.\.]\d{1,2}[/\-\.\.]\d{2,4}\b',
-        r'\b\d{1,2}(:st|nd|rd|th)?\s+(:Jan(?:uary)?|Feb(?:bruary)?|Mar(?:ch)?|Apr(?:il)?|May|Jun(?:e)?|Jul(?:y)?|
23    Aug(?:ust)?|Sep(?:tember)?|Oct(?:ober)?|Nov(?:ember)?|Dec(?:ember)?)\s+d{4}\b',
7        r'\b(?:Jan(?:uary)?|Feb(?:bruary)?|Mar(?:ch)?|Apr(?:il)?|May|Jun(?:e)?|Jul(?:y)?|Aug(?:ust)?|Sep(?:tember)?|
23    Oct(?:ober)?|Nov(?:ember)?|Dec(?:ember)?)\s+d{1,2}(:st|nd|rd|th)?\s+d{4}\b',
-        r'\b\d{4}[-\/]\d{2}[-\/]\d{2}\b',
23        r'\btoday\b|yesterday\b|btomorrow\b',
8        r'\b(?:last|this|next)\s+(:week|month|year|Monday|Tuesday|Wednesday|Thursday|Friday|Saturday|Sunday)\b',
-        r'\b\d+s+(:days?|weeks?|months?|years?)\s+ago\b',
23        r'\b(?:in|since|from)\s+(:January|February|March|April|May|June|July|August|September|October|November|December)
9        \s+d{4}\b',
-    ]
24    for pat in date_patterns:
0        for m in re.finditer(pat, text, re.IGNORECASE):
-            if no_overlap(m.start(), m.end()):
24                entities.append({"type": "DATE", "text": m.group(), "start": m.start(), "end": m.end(), "confidence": 0.95})
1            seen_spans.append((m.start(), m.end()))
24
2
-     # — DOSAGES —————
24    dosage_patterns = [
3        r'\b\d+(?:\.\d+)?\s*(?:mg|mcg|µg|micrograms?|g|mL|mL|L|mmol|units?|IU|mEq|mg/kg|mcg/kg|mg/mL|µg/kg)\b',

```

```

-         r'\b\d+\.s*(?:micrograms?|milligrams?|grams?)\b',
24         r'\b(?:once|twice|three\s+times|four\s+times)\s+(?:daily|a\s+day|per\s+day)\b',
4         r'\b(?:BD|TDS|QDS|OD|PRN|QID|TID|BID|OM|ON|stat|STAT|nocte)\b',
-         r'\b\d+\.s*(?:tablet|capsule|puff|drop|patch|vial|ampoule|sachet)s?\b',
24         r'\b(?:loading dose|maintenance dose|starting dose|initial dose)\b',
5         r'\b\d+\.s*(?:\.\d+)?%s*(?:solution|cream|ointment|gel|drops?|lotion|infusion)\b',
-     ]
24     for pat in dosage_patterns:
6         for m in re.finditer(pat, text, re.IGNORECASE):
-             if no_overlap(m.start(), m.end()):
24                 entities.append({"type": "DOSAGE", "text": m.group(), "start": m.start(), "end": m.end(), "confidence": 0.9
7 2})
-                 seen_spans.append((m.start(), m.end()))
24
8     # — VITAL SIGNS —————
vital_patterns = [
24         r'\bBP\s*?:\s*\d{2,3}\s*/\s*\d{2,3}\b',
9         r'\b(?:blood\s+pressure)\s*?:\s*\d{2,3}\s*/\s*\d{2,3}\b',
-         r'\b(?:HR|heart\s+rate|pulse\s+rate|pulse)\s*?:\s*\d{2,3}\s*(?:bpm|/min)?\b',
25         r'\bSpO2\s*?:\s*\d{2,3}\s*%\b',
0         r'\b(?:O2\s+sat(?:uration)?|oxygen\s+saturation)\s*?:\s*\d{2,3}\s*%\b',
-         r'\b(?:temp(?:erature)?)\s*?:\s*\d{2}(\.\d)?\s*°\s*[CF]?\b',
25         r'\b(?:RR|respiratory\s+rate)\s*?:\s*\d{1,2}\b',
1         r'\bGCS\s*?:\s*\d{1,2}(\./15)?\b',
-         r'\bBMI\s*?:\s*\d{2}(\.\d+)?\b',
25         r'\b(?:weight|Wt)\s*?:\s*\d+(\.\d+)?\s*kg\b',
2         r'\b(?:height|Ht)\s*?:\s*\d+(\.\d+)?\s*(?:cm|m)\b',
-     ]
25     for pat in vital_patterns:
3         for m in re.finditer(pat, text, re.IGNORECASE):
-             if no_overlap(m.start(), m.end()):
25                 entities.append({"type": "VITAL_SIGN", "text": m.group(), "start": m.start(), "end": m.end(),
4 "confidence": 0.96})
-                 seen_spans.append((m.start(), m.end()))
25
5     # — LAB VALUES —————
lab_patterns = [
25         r'\b(?:HbA1c|haemoglobin\s+A1c)\s*?:\s*\d+(\.\d+)?\s*%\b',
6         r'\b(?:Hb|haemoglobin)\s*?:\s*\d+(\.\d+)?\s*(?:g/dL|g/L)?\b',
-         r'\b(?:Na|sodium)\s*?:\s*\d{2,3}\s*(?:mmol/L)?\b',
25         r'\b(?:K|potassium)\s*?:\s*\d(\.\d)?\s*(?:mmol/L)?\b',
7         r'\b(?:Cr|creatinine)\s*?:\s*\d+(\.\d+)?\s*(?:µmol/L|umol/L|mg/dL)?\b',
-         r'\b(?:eGFR|GFR)\s*?:\s*\d+(\.\d+)?\b',
25         r'\b(?:WBC|white\s+(?:blood\s+)?cell\s+count)\s*?:\s*\d+(\.\d+)?\b',
8         r'\b(?:Hct|haematocrit)\s*?:\s*\d+(\.\d+)?%\b',
-         r'\b(?:PLT|platelets)\s*?:\s*\d+(\.\d+)?\b',
25         r'\b(?:ALT|AST|ALP|GGT|bilirubin)\s*?:\s*\d+(\.\d+)?\s*(?:U/L|IU/L|µmol/L)?\b',
9         r'\b(?:TSH|free\s+T4|T3)\s*?:\s*\d+(\.\d+)?\b',
-         r'\b(?:CRP|ESR|procalcitonin)\s*?:\s*\d+(\.\d+)?\b',
26         r'\b(?:troponin|BNP|NT-proBNP)\s*?:\s*\d+(\.\d+)?\b',
0         r'\b(?:INR|APTT|PT)\s*?:\s*\d+(\.\d+)?\b',
-         r'\b(?:cholesterol|LDL|HDL|triglycerides)\s*?:\s*\d+(\.\d+)?\s*(?:mmol/L)?\b',
26         r'\b(?:glucose|blood\s+glucose|fasting\s+glucose)\s*?:\s*\d+(\.\d+)?\s*(?:mmol/L|mg/dL)?\b',
1         r'\b(?:urea|BUN|uric\s+acid)\s*?:\s*\d+(\.\d+)?\b',
-         r'\b(?:albumin|total\s+protein)\s*?:\s*\d+(\.\d+)?\s*(?:g/L|g/dL)?\b',
26         r'\b(?:calcium|phosphate|magnesium)\s*?:\s*\d+(\.\d+)?\s*(?:mmol/L)?\b',
2         r'\b(?:ferritin|iron|TIBC|transferrin\s+saturation)\s*?:\s*\d+(\.\d+)?\b',
-     ]
26     for pat in lab_patterns:
3         for m in re.finditer(pat, text, re.IGNORECASE):
-             if no_overlap(m.start(), m.end()):
26                 entities.append({"type": "LAB_VALUE", "text": m.group(), "start": m.start(), "end": m.end(), "confidence":
4 0.94})
-                 seen_spans.append((m.start(), m.end()))
26
5     # — MEDICATIONS —————
for med in sorted(_NLP_MEDICATIONS, key=len, reverse=True):
26         pat = r'\b' + re.escape(med) + r'\b'
6         for m in re.finditer(pat, text, re.IGNORECASE):
-             if no_overlap(m.start(), m.end()):
26                 entities.append({"type": "MEDICATION", "text": m.group(), "start": m.start(), "end": m.end(),
7 "confidence": 0.90})
-                 seen_spans.append((m.start(), m.end()))
26
8     # — DIAGNOSES —————
for diag in sorted(_NLP_DIAGNOSES, key=len, reverse=True):
26         pat = r'\b' + re.escape(diag) + r'\b'
9         for m in re.finditer(pat, text, re.IGNORECASE):
-             if no_overlap(m.start(), m.end()):

```

```

27         entities.append({"type": "DIAGNOSIS", "text": m.group(), "start": m.start(), "end": m.end(), "confidence":
0 0.88})
27         seen_spans.append((m.start(), m.end()))
1
27         entities.sort(key=lambda x: x["start"])
27
27         counts: dict[str, int] = {}
2         for e in entities:
27             counts[e["type"]] = counts.get(e["type"], 0) + 1
27
27         return {"entities": entities, "counts": counts, "total": len(entities)}
3
27
4 # — Voice Clinical Notes —————
27
27 def format_clinical_note(transcript: str) -> str:
5     """Format a voice transcript into a structured clinical note using NLP entity extraction."""
27     from datetime import datetime
27
27     # Strip filler words common in dictation
6     fillers = r'\b(um|uh|er|ah|hmm|so|like|you know|sort of|kind of|basically|right|okay so|and so)\b'
27     text = re.sub(fillers, '', transcript, flags=re.IGNORECASE)
27     text = re.sub(r'\s{2,}', ' ', text).strip()
7
27     # Split into sentences (handle lack of punctuation in speech-to-text)
8     # Split on ". " or "? " or "! " or common verbal full-stops like ", so " / ", and "
27     raw_sents = re.split(r'(?<=[!?.])\s+|(?<=\n)', text)
27     sentences = [s.strip() for s in raw_sents if len(s.strip()) > 3]
9
27     # Extract entities from full text
28     nlp = _run_nlp_extractor(text)
0     def ents(t): return list(dict.fromkeys(e['text'] for e in nlp['entities'] if e['type'] == t))
28     meds = ents('MEDICATION')
28     diags = ents('DIAGNOSIS')
1     vitals = ents('VITAL_SIGN')
28     labs = ents('LAB_VALUE')
28     doses = ents('DOSAGE')
2
27     # Keyword patterns for section classification (use alternation groups, not char classes)
28     PC = re.compile(r'\b(presenting|presented|presents|complaining|complaint|came in|brought in|attending|attended|
3     referr(?:ed|al)?|admitted|c/o|chief complaint)\b', re.I)
27     HX = re.compile(r'\b(background|history of|known|past medical|pmh|previous|chronic|suffered|diagnosed with|has a
28     history|long.?standing)\b', re.I)
4     EX = re.compile(r'\b(examin(?:ation|ed|es)?|on exam(?:ination)?|inspection|auscultation|palpation|found on|on
27     observation|looks|appears|clinically)\b', re.I)
28     IX = re.compile(r'\b(bloods?(?:s+?(?:test|show|result))|blood\s+test|ecg|x.?ray|ct\s+scan|mri|ultrasound|imaging|
5     scan(?:ned)?|result(?:s)?|showed|pending|investig(?:ations)?|ated)|fbc|u&e|lft|tfts?|troponin|cultures?)\b', re.I)
27     AS = re.compile(r'\b(impression|assessment|working\s+diagnosis|likely|probable|suspect|rule\s+out|differential|my\s+
28     (?:working\s+)?impression|diagnosis is|consistent with)\b', re.I)
6     PL = re.compile(r'\b(plan(?:ning)?(?:s+for)?|for\s+(?:serial|urgent|routine|review)|refer(?:al)?|discharge|
27     prescrib(?:e|ing)|start(?:ing)?|stop(?:ping)?|continue|follow.?up|review|arrange|request|monitor|titrat(?:e|ing)?|commence)
28     \b', re.I)
7
27     # Classify sentences into sections
28     sections: dict[str, list[str]] = {
8         'PRESENTING COMPLAINT': [],
27         'HISTORY': [],
28         'EXAMINATION': [],
9         'INVESTIGATIONS': [],
27         'ASSESSMENT': [],
29         'PLAN': [],
0         'OTHER': [],
27     }
29
1     vital_lower = [v.lower() for v in vitals]
27     lab_lower = [l.lower() for l in labs]
29
2     for sent in sentences:
27         sl = sent.lower()
29         has_vital = any(v in sl for v in vital_lower)
3         has_lab = any(l in sl for l in lab_lower)
27
29         if AS.search(sent):
4             sections['ASSESSMENT'].append(sent)
27         elif PL.search(sent) and not EX.search(sent):
29             sections['PLAN'].append(sent)
27         elif IX.search(sent) or has_lab:
5             sections['INVESTIGATIONS'].append(sent)
27         elif EX.search(sent) or has_vital:
29

```

```

6         sections['EXAMINATION'].append(sent)
elif PC.search(sent):
29     sections['PRESENTING COMPLAINT'].append(sent)
7     elif HX.search(sent):
        sections['HISTORY'].append(sent)
29     else:
8         # Fallback: first unclassified sentence → PC; rest → History
        if not sections['PRESENTING COMPLAINT']:
29             sections['PRESENTING COMPLAINT'].append(sent)
9         else:
            sections['OTHER'].append(sent)
30
0     # Promote orphaned vital-containing sentences in OTHER to EXAMINATION
    for s in list(sections['OTHER']):
30         if any(v in s.lower() for v in vital_lower) or any(l in s.lower() for l in lab_lower):
1             sections['INVESTIGATIONS' if any(l in s.lower() for l in lab_lower) else 'EXAMINATION'].append(s)
            sections['OTHER'].remove(s)
30
2     # Build formatted output
    now = datetime.now()
30     lines = [
3         "CLINICAL NOTE",
        f"Date: {now.strftime('%-d %B %Y')}",
30         f"Time: {now.strftime('%H:%M')}",
4         "",
        ]
30
5     section_order = ['PRESENTING COMPLAINT', 'HISTORY', 'EXAMINATION', 'INVESTIGATIONS', 'ASSESSMENT', 'PLAN']
    for section in section_order:
30         sents = sections[section]
6         if sents:
            lines.append(f"{section}:")
            for s in sents:
30                 # Capitalise first letter
                lines.append(f" {s[0].upper() + s[1:] if s else ''}")
            lines.append("")
30
    if sections['OTHER']:
30         lines.append("ADDITIONAL NOTES:")
9         for s in sections['OTHER']:
            lines.append(f" {s}")
31         lines.append("")
0
    # Structured medication list (deduplicated, from NLP)
31     if meds:
1         lines.append("MEDICATIONS MENTIONED:")
        for m in meds:
31             # Try to find associated dose immediately after the med name in text
2             idx = text.lower().find(m.lower())
            dose_context = ""
31             if idx >= 0:
3                 after = text[idx + len(m):idx + len(m) + 30]
                dose_m = re.search(r'\s*(\d+\s*(?:mg|mcg|g|ml)\s*(?:BD|TDS|QDS|OD|PRN|once daily|twice daily)?)', after,
31 re.I)
4                 if dose_m:
                    dose_context = f" {dose_m.group(1).strip()}"
31                 lines.append(f" - {m.title()}{dose_context}")
5                 lines.append("")
31
    # Diagnoses mentioned
6     if diags:
        lines.append("DIAGNOSES / CONDITIONS:")
31         for d in diags:
7             lines.append(f" - {d.title()}")
            lines.append("")
31
8     result = '\n'.join(lines).rstrip()
    return result
31
9
@app.route("/voice-notes")
32 def voice_notes():
0     return render_template("voice_notes.html")
32
1 @app.route("/voice-notes/format", methods=["POST"])
def voice_notes_format():
32     try:
2         data = request.get_json(force=True) or {}

```

```

-         transcript = data.get("transcript", "").strip()
32     if not transcript:
3         return jsonify({"ok": False, "error": "No transcript provided"}), 400
-         note = _format_clinical_note(transcript)
32     return jsonify({"ok": True, "note": note})
4     except Exception as exc:
-         return jsonify({"ok": False, "error": str(exc)}), 500
32
5
@app.route("/nlp-extractor")
32 def nlp_extractor():
6     return render_template("nlp_extractor.html")
32
7 @app.route("/nlp-extractor/run", methods=["POST"])
def nlp_extractor_run():
32     try:
8         data = request.get_json(force=True) or {}
-         text = data.get("text", "").strip()
32     if not text:
9         return jsonify({"ok": False, "error": "No text provided"}), 400
-         result = _run_nlp_extractor(text)
33     return jsonify({"ok": True, **result})
0     except Exception as exc:
33     return jsonify({"ok": False, "error": str(exc)}), 500
33
1
# — Drug Interaction Checker —————
33
2 DRUG_ALIASES: dict[str, str] = {
-     "acetaminophen": "paracetamol", "tylenol": "paracetamol",
33     "advil": "ibuprofen", "nurofen": "ibuprofen", "brufen": "ibuprofen",
3     "voltaren": "diclofenac", "voltarol": "diclofenac",
-     "coumadin": "warfarin",
33     "lasix": "furosemide", "frusemide": "furosemide",
4     "zocor": "simvastatin",
-     "lipitor": "atorvastatin",
33     "crestor": "rosuvastatin",
5     "glucophage": "metformin",
-     "lanoxin": "digoxin",
33     "cordarone": "amiodarone",
6     "plavix": "clopidogrel",
-     "eliquis": "apixaban",
33     "xarelto": "rivaroxaban",
7     "pradaxa": "dabigatran",
-     "prozac": "fluoxetine",
33     "zoloft": "sertraline",
8     "lexapro": "escitalopram",
-     "celexa": "citalopram",
33     "effexor": "venlafaxine",
9     "cymbalta": "duloxetine",
-     "seroquel": "quetiapine",
34     "risperdal": "risperidone",
0     "zyprexa": "olanzapine",
-     "depakote": "valproate", "depakene": "valproate",
34     "sodium valproate": "valproate", "semisodium valproate": "valproate",
1     "tegretol": "carbamazepine",
-     "lamictal": "lamotrigine",
34     "neurontin": "gabapentin",
2     "lyrica": "pregabalin",
-     "diflucan": "fluconazole",
34     "flagyl": "metronidazole",
3     "rifadin": "rifampicin", "rifampin": "rifampicin",
-     "sandimmun": "ciclosporin", "cyclosporine": "ciclosporin",
34     "ciclosporine": "ciclosporin", "cyclosporin": "ciclosporin",
4     "imuran": "azathioprine",
-     "zyloprim": "allopurinol",
34     "nitro": "nitroglycerine", "gtn": "nitroglycerine",
5     "glyceryl trinitrate": "nitroglycerine",
-     "viagra": "sildenafil",
34     "cialis": "tadalafil",
6     "prilosec": "omeprazole",
-     "nexium": "esomeprazole",
34     "clexane": "enoxaparin", "lovenox": "enoxaparin",
7     "zithromax": "azithromycin",
-     "biacin": "clarithromycin",
34     "cipro": "ciprofloxacin",
8     "asa": "aspirin", "acetylsalicylic acid": "aspirin",
-     "ms contin": "morphine", "oramorph": "morphine",

```

```

34 "oxycontin": "oxycodone",
9 "ultram": "tramadol",
  "colcrys": "colchicine",
35 "inderal": "propranolol", "avlocardyl": "propranolol",
0 "cardizem": "diltiazem",
  "calan": "verapamil", "isoptin": "verapamil",
35 "slo-phyllin": "theophylline",
1 }

35 DRUG_INTERACTIONS: list[dict] = [
2 # — CONTRAINDICATED —————
  {
35 "drugs": ["selegiline", "sertraline"], "severity": "CONTRAINDICATED",
3 "mechanism": "MAOI + SSRI: dual serotonergic stimulation causing serotonin syndrome.",
  "effect": "Potentially fatal serotonin syndrome: hyperthermia, rigidity, clonus, rhabdomyolysis, seizures.",
35 "recommendation": "Do not co-prescribe. Allow ≥14 days washout after stopping MAOI before starting SSRI.",
4 "onset": "Hours to days"
  },
35 {
5 "drugs": ["selegiline", "fluoxetine"], "severity": "CONTRAINDICATED",
  "mechanism": "MAOI + SSRI. Fluoxetine's active metabolite has a 1–2 week half-life, extending the washout
35 requirement.",
6 "effect": "Potentially fatal serotonin syndrome.",
  "recommendation": "Allow ≥5 weeks washout after stopping fluoxetine before starting any MAOI.",
35 "onset": "Hours to days"
7 },
35 {
8 "drugs": ["selegiline", "venlafaxine"], "severity": "CONTRAINDICATED",
  "mechanism": "MAOI + SNRI: dual serotonergic stimulation.",
  "effect": "Potentially fatal serotonin syndrome.",
35 "recommendation": "Contraindicated. ≥14 days washout after stopping MAOI.",
9 "onset": "Hours to days"
  },
36 {
0 "drugs": ["selegiline", "tramadol"], "severity": "CONTRAINDICATED",
  "mechanism":
36 "Tramadol inhibits serotonin and noradrenaline reuptake; combined with MAO inhibition causes serotonin syndrome.",
1 "effect": "Serotonin syndrome; also lowers seizure threshold.",
  "recommendation": "Contraindicated. Use alternative analgesia (e.g., paracetamol, codeine).",
36 "onset": "Hours"
2 },
36 {
3 "drugs": ["rasagiline", "sertraline"], "severity": "CONTRAINDICATED",
  "mechanism": "Selective MAO-B inhibitor + SSRI: serotonin syndrome.",
  "effect": "Serotonin syndrome.",
36 "recommendation": "Contraindicated. Use alternative antidepressant (e.g., mirtazapine).",
4 "onset": "Hours to days"
  },
36 {
5 "drugs": ["rasagiline", "tramadol"], "severity": "CONTRAINDICATED",
  "mechanism": "MAO-B inhibitor + tramadol serotonin reuptake inhibition.",
36 "effect": "Serotonin syndrome; seizures.",
6 "recommendation": "Contraindicated. Use alternative analgesia.",
36 "onset": "Hours"
7 },
36 {
8 "drugs": ["sildenafil", "nitroglycerine"], "severity": "CONTRAINDICATED",
  "mechanism": "Both increase cGMP via the nitric oxide pathway; additive smooth muscle vasodilation.",
  "effect": "Severe, potentially fatal hypotension.",
36 "recommendation": "Contraindicated within 24 hours of sildenafil. Never co-prescribe as regular medications.",
9 "onset": "Minutes"
  },
37 {
0 "drugs": ["tadalafil", "nitroglycerine"], "severity": "CONTRAINDICATED",
  "mechanism": "Additive cGMP-mediated vasodilation. Tadalafil half-life is 17–35 hours.",
  "effect": "Severe hypotension.",
37 "recommendation": "Contraindicated within 48 hours of tadalafil.",
1 "onset": "Minutes"
  },
37 {
2 "drugs": ["sildenafil", "isosorbide"], "severity": "CONTRAINDICATED",
  "mechanism": "PDE5 inhibitor + nitrate: additive vasodilation via cGMP pathway.",
37 "effect": "Severe hypotension.",
3 "recommendation": "Contraindicated.",
  "onset": "Minutes to hours"
37 },
4 {
  "drugs": ["tadalafil", "isosorbide"], "severity": "CONTRAINDICATED",
37 "mechanism": "PDE5 inhibitor + nitrate.",

```

```

5      "effect": "Severe hypotension.",
    -   "recommendation": "Contraindicated.",
37     "onset": "Minutes to hours"
6   },
    -   {
37     "drugs": ["linezolid", "sertraline"], "severity": "CONTRAINDICATED",
7     "mechanism": "Linezolid is a reversible, non-selective MAO inhibitor; combined with SSRIs causes serotonin
syndrome.",
37     "effect": "Serotonin syndrome.",
8     "recommendation": "Avoid. Use alternative antibiotic. If linezolid essential, discontinue SSRI ≥2 weeks
beforehand.",
37     "onset": "Hours to days"
9   },
    -   {
38     "drugs": ["linezolid", "fluoxetine"], "severity": "CONTRAINDICATED",
0     "mechanism": "Linezolid MAO inhibition + SSRI with long-acting active metabolite.",
38     "effect": "Serotonin syndrome.",
38     "recommendation": "Contraindicated. ≥5 weeks washout after fluoxetine.",
1     "onset": "Hours to days"
    -   },
38     {
2     "drugs": ["linezolid", "venlafaxine"], "severity": "CONTRAINDICATED",
38     "mechanism": "Linezolid MAO inhibition + SNRI.",
38     "effect": "Serotonin syndrome.",
3     "recommendation": "Contraindicated.",
    -   "onset": "Hours to days"
38   },
4     {
38     "drugs": ["linezolid", "tramadol"], "severity": "CONTRAINDICATED",
5     "mechanism": "Linezolid MAO inhibition + tramadol serotonin/noradrenaline reuptake inhibition.",
38     "effect": "Serotonin syndrome.",
38     "recommendation": "Contraindicated. Use alternative analgesia.",
6     "onset": "Hours"
    -   },
38   # — MAJOR —————
7   {
38     "drugs": ["warfarin", "amiodarone"], "severity": "MAJOR",
    -     "mechanism": "Amiodarone inhibits CYP2C9 (primary warfarin metabolising enzyme) and CYP3A4.",
38     "effect": "INR increases 2–3× within days to weeks; major haemorrhage risk.",
8     "recommendation": "Reduce warfarin dose by 30–50% on starting amiodarone. Monitor INR every 3–5 days until stable,
then weekly for 4–6 weeks.",
38     "onset": "Days to weeks"
9   },
    -   {
39     "drugs": ["warfarin", "metronidazole"], "severity": "MAJOR",
0     "mechanism": "Metronidazole inhibits CYP2C9 and may directly inhibit warfarin metabolism.",
39     "effect": "Elevated INR; bleeding risk.",
39     "recommendation": "Monitor INR during and for 7–10 days after metronidazole. Empirical warfarin dose reduction of
1 ~25% often needed.",
    -     "onset": "2–7 days"
39   },
2     {
39     "drugs": ["warfarin", "fluconazole"], "severity": "MAJOR",
    -     "mechanism": "Fluconazole inhibits CYP2C9 (major) and CYP3A4 (minor).",
39     "effect": "Marked INR elevation; serious bleeding risk.",
3     "recommendation": "Check INR on day 3 of fluconazole. Empirical warfarin dose reduction 30–50% often required.",
39     "onset": "2–4 days"
4   },
    -   {
39     "drugs": ["warfarin", "aspirin"], "severity": "MAJOR",
5     "mechanism": "Additive: COX-1 antiplatelet inhibition + anticoagulant + GI mucosal damage.",
39     "effect": "Major bleeding, particularly GI haemorrhage.",
39     "recommendation": "Avoid unless dual antithrombotic therapy is specifically indicated (ACS with AF). Always add
6 PPI.",
    -     "onset": "Immediate"
39   },
7     {
39     "drugs": ["warfarin", "ibuprofen"], "severity": "MAJOR",
    -     "mechanism": "NSAIDs inhibit platelet aggregation and cause GI mucosal damage; potential CYP2C9 competition.",
39     "effect": "Significantly increased GI and systemic bleeding risk.",
8     "recommendation": "Avoid. Use paracetamol. If NSAID essential, monitor INR closely and add PPI.",
39     "onset": "Days"
9   },
    -   {
40     "drugs": ["warfarin", "naproxen"], "severity": "MAJOR",
0     "mechanism": "NSAID: antiplatelet + gastric mucosal damage.",
    -     "effect": "Increased bleeding risk.",
40     "recommendation": "Avoid. Paracetamol preferred.",
1     "onset": "Days"

```

```

-     },
40    {
2      "drugs": ["warfarin", "diclofenac"], "severity": "MAJOR",
-      "mechanism": "NSAID + CYP2C9 inhibition by diclofenac.",
40      "effect": "Increased bleeding risk.",
3      "recommendation": "Avoid. Paracetamol preferred.",
-      "onset": "Days"
40    },
4    {
-      "drugs": ["warfarin", "carbamazepine"], "severity": "MAJOR",
40      "mechanism": "Carbamazepine is a potent CYP2C9/CYP3A4 inducer, accelerating warfarin clearance.",
5      "effect": "Reduced anticoagulation; thrombosis risk.",
-      "recommendation": "Monitor INR closely on starting carbamazepine; warfarin dose may need to increase
40 significantly.",
6      "onset": "1-2 weeks"
-    },
40    {
7      "drugs": ["warfarin", "rifampicin"], "severity": "MAJOR",
-      "mechanism": "Rifampicin is one of the most potent CYP enzyme inducers (CYP2C9, CYP3A4, CYP1A2).",
40      "effect": "Dramatic reduction in warfarin effect; INR may fall to sub-therapeutic levels.",
8      "recommendation": "Warfarin dose may need to double or more. Monitor INR very frequently. Consider alternative
- anticoagulation.",
40      "onset": "Days to 2 weeks"
9    },
-    {
41      "drugs": ["digoxin", "amiodarone"], "severity": "MAJOR",
0      "mechanism": "Amiodarone inhibits P-glycoprotein, reducing digoxin renal excretion and altering volume of
41 distribution.",
1      "effect": "Digoxin toxicity: nausea, bradycardia, AV block, ventricular arrhythmias.",
-      "recommendation": "Reduce digoxin dose by 50% on starting amiodarone. Monitor digoxin level and ECG closely.",
41      "onset": "Days"
2    },
-    {
41      "drugs": ["digoxin", "verapamil"], "severity": "MAJOR",
3      "mechanism": "Verapamil inhibits P-glycoprotein, reducing digoxin renal clearance; additive AV nodal depression.",
-      "effect": "Digoxin toxicity and bradycardia/AV block.",
41      "recommendation": "Reduce digoxin dose by 33-50%. Monitor ECG and digoxin levels.",
4      "onset": "Days"
-    },
41    {
5      "drugs": ["digoxin", "diltiazem"], "severity": "MAJOR",
-      "mechanism": "Diltiazem reduces digoxin clearance and has additive AV nodal effects.",
41      "effect": "Elevated digoxin levels; bradycardia; AV block.",
6      "recommendation": "Monitor digoxin levels and ECG. Reduce digoxin dose accordingly.",
-      "onset": "Days"
41    },
7    {
-      "drugs": ["methotrexate", "ibuprofen"], "severity": "MAJOR",
41      "mechanism": "NSAIDs reduce renal prostaglandin synthesis, decreasing GFR and methotrexate renal excretion; may
also displace methotrexate from plasma proteins.",
8      "effect": "Methotrexate toxicity: pancytopenia, mucositis, hepatotoxicity, nephrotoxicity.",
-      "recommendation":
41 "Avoid NSAIDs in patients on methotrexate. Use paracetamol. If unavoidable, monitor FBC and renal function closely.",
9      "onset": "Days"
-    },
42    {
0      "drugs": ["methotrexate", "naproxen"], "severity": "MAJOR",
-      "mechanism": "NSAID-mediated reduction in methotrexate renal clearance.",
42      "effect": "Methotrexate toxicity.",
1      "recommendation": "Avoid. Use paracetamol.",
-      "onset": "Days"
42    },
2    {
-      "drugs": ["methotrexate", "trimethoprim"], "severity": "MAJOR",
42      "mechanism": "Both are antifolates; additive inhibition of dihydrofolate reductase.",
3      "effect": "Severe pancytopenia, megaloblastic anaemia.",
-      "recommendation": "Avoid. If unavoidable, folinic acid rescue and close FBC monitoring.",
42      "onset": "Days"
4    },
-    {
42      "drugs": ["methotrexate", "co-trimoxazole"], "severity": "MAJOR",
5      "mechanism": "Co-trimoxazole = trimethoprim (antifolate) + sulfamethoxazole (reduces methotrexate renal
42 excretion).",
6      "effect": "Severe antifolate toxicity: pancytopenia, mucositis.",
-      "recommendation": "Avoid. If essential, folinic acid prophylaxis and close FBC monitoring required.",
42      "onset": "Days"
7    },
-    {
-      "drugs": ["lithium", "ibuprofen"], "severity": "MAJOR",

```

```

42     "mechanism": "NSAIDs reduce renal prostaglandin synthesis, decreasing GFR and lithium renal excretion.",
8     "effect": "Lithium toxicity: tremor, ataxia, confusion, renal failure, cardiac arrhythmias.",
    "recommendation": "Avoid NSAIDs in patients on lithium. Use paracetamol. If NSAID unavoidable, reduce lithium dose
42 and monitor levels closely.",
9     "onset": "Days"
    },
43 {
0     "drugs": ["lithium", "naproxen"], "severity": "MAJOR",
    "mechanism": "NSAID: reduced GFR → reduced lithium excretion.",
43     "effect": "Lithium toxicity.",
1     "recommendation": "Avoid. Use paracetamol.",
    "onset": "Days"
43 },
2 {
    "drugs": ["lithium", "diclofenac"], "severity": "MAJOR",
43     "mechanism": "NSAID: reduced lithium excretion.",
3     "effect": "Lithium toxicity.",
    "recommendation": "Avoid.",
43     "onset": "Days"
4 },
43 {
    "drugs": ["lithium", "ramipril"], "severity": "MAJOR",
5     "mechanism": "ACE inhibitors reduce aldosterone-mediated sodium reabsorption, causing compensatory lithium
retention in the proximal tubule.",
43     "effect": "Lithium toxicity even at previously therapeutic doses.",
6     "recommendation": "Monitor lithium levels closely when starting or changing ACE inhibitor dose. Consider empirical
50% lithium dose reduction.",
43     "onset": "Days to weeks"
7 },
    {
43     "drugs": ["lithium", "lisinopril"], "severity": "MAJOR",
8     "mechanism": "ACE inhibitor: sodium depletion → compensatory lithium reabsorption.",
    "effect": "Lithium toxicity.",
43     "recommendation": "Monitor lithium levels closely.",
9     "onset": "Days to weeks"
44 },
0 {
    "drugs": ["lithium", "furosemide"], "severity": "MAJOR",
44     "mechanism": "Loop diuretics cause sodium depletion, triggering compensatory lithium reabsorption in the proximal
1 tubule.",
    "effect": "Lithium toxicity.",
44     "recommendation": "Monitor lithium levels closely. Ensure adequate dietary sodium intake.",
2     "onset": "Days"
44 },
    {
44     "drugs": ["morphine", "diazepam"], "severity": "MAJOR",
3     "mechanism": "Additive CNS and respiratory depression (opioid receptor agonism + GABA-A positive allosteric
modulation).",
44     "effect":
4     "Respiratory depression, coma, death. This combination accounts for a large proportion of opioid overdose deaths.",
    "recommendation": "Avoid where possible. If co-prescribed (palliative care), use lowest doses, monitor respiratory
44 rate closely, and have naloxone available.",
5     "onset": "Hours"
44 },
6 {
    "drugs": ["oxycodone", "diazepam"], "severity": "MAJOR",
44     "mechanism": "Additive opioid + benzodiazepine respiratory depression.",
    "effect": "Respiratory depression, sedation.",
7     "recommendation": "Avoid where possible. Naloxone should be accessible if co-prescribed.",
44     "onset": "Hours"
8 },
    {
44     "drugs": ["codeine", "diazepam"], "severity": "MAJOR",
    "mechanism": "Additive respiratory depression (opioid + benzodiazepine).",
9     "effect": "Respiratory depression.",
    "recommendation": "Avoid. If necessary, lowest doses and close monitoring.",
45     "onset": "Hours"
0 },
    {
45     "drugs": ["tramadol", "sertraline"], "severity": "MAJOR",
1     "mechanism": "Tramadol inhibits serotonin and noradrenaline reuptake; combined with SSRIs substantially raises
synaptic serotonin.",
45     "effect": "Serotonin syndrome: agitation, confusion, hyperthermia, tachycardia, clonus, rigidity.",
2     "recommendation":
    "Avoid if possible. If co-prescribed, use lowest tramadol dose and monitor for serotonin syndrome signs.",
45     "onset": "Hours to days"
3 },
45 {
    "drugs": ["tramadol", "fluoxetine"], "severity": "MAJOR",

```

```

4      "mechanism": "Serotonin syndrome + fluoxetine inhibits CYP2D6 (tramadol metabolic activation).",
    -   "effect": "Serotonin syndrome.",
45    "recommendation": "Avoid. Use alternative analgesia.",
5    "onset": "Hours"
    - },
45    {
6      "drugs": ["tramadol", "citalopram"], "severity": "MAJOR",
    -   "mechanism": "Serotonin syndrome (SSRI + tramadol serotonin reuptake inhibition).",
45    "effect": "Serotonin syndrome.",
7    "recommendation": "Avoid.",
    -   "onset": "Hours"
45    },
8    {
    -   "drugs": ["tramadol", "escitalopram"], "severity": "MAJOR",
45    "mechanism": "Serotonin syndrome (SSRI + tramadol).",
9    "effect": "Serotonin syndrome.",
    -   "recommendation": "Avoid.",
46    "onset": "Hours"
0    },
46    {
1      "drugs": ["tramadol", "venlafaxine"], "severity": "MAJOR",
    -   "mechanism": "Serotonin syndrome (SNRI + tramadol); also additive seizure risk.",
46    "effect": "Serotonin syndrome; seizures.",
2    "recommendation": "Avoid. Use alternative analgesia.",
    -   "onset": "Hours"
46    },
3    {
    -   "drugs": ["metoprolol", "verapamil"], "severity": "MAJOR",
46    "mechanism": "Additive AV nodal depression: beta-blockade + rate-limiting calcium channel blockade.",
46    "effect": "Severe bradycardia, complete heart block.",
4    "recommendation": "Avoid. Use dihydropyridine CCB (amlodipine) instead of verapamil if both rate control and BP
    -   are needed.",
46    "onset": "Hours"
5    },
46    {
6      "drugs": ["bisoprolol", "verapamil"], "severity": "MAJOR",
    -   "mechanism": "Additive AV nodal depression.",
46    "effect": "Bradycardia, heart block.",
    -   "recommendation": "Avoid. Use amlodipine if CCB required.",
7    "onset": "Hours"
    - },
46    {
8      "drugs": ["atenolol", "verapamil"], "severity": "MAJOR",
    -   "mechanism": "Additive AV nodal depression.",
46    "effect": "Bradycardia, heart block.",
9    "recommendation": "Avoid.",
    -   "onset": "Hours"
47    },
0    {
    -   "drugs": ["propranolol", "verapamil"], "severity": "MAJOR",
47    "mechanism": "Additive AV nodal depression.",
1    "effect": "Bradycardia, heart block.",
    -   "recommendation": "Avoid.",
47    "onset": "Hours"
2    },
47    {
3      "drugs": ["metoprolol", "diltiazem"], "severity": "MAJOR",
    -   "mechanism": "Additive AV nodal depression (beta-blocker + rate-limiting CCB).",
47    "effect": "Severe bradycardia, heart block.",
    -   "recommendation": "Avoid. Use amlodipine instead of diltiazem if CCB needed.",
4    "onset": "Hours"
    - },
47    {
5      "drugs": ["bisoprolol", "diltiazem"], "severity": "MAJOR",
    -   "mechanism": "Additive AV nodal depression.",
47    "effect": "Bradycardia, heart block.",
6    "recommendation": "Avoid. Use amlodipine.",
    -   "onset": "Hours"
47    },
7    {
    -   "drugs": ["amiodarone", "sotalol"], "severity": "MAJOR",
47    "mechanism": "Additive QTc prolongation via multiple mechanisms (IKr blockade + multiple ion channel effects).",
8    "effect": "Torsades de pointes; ventricular fibrillation.",
    -   "recommendation": "Avoid combination. ECG monitoring mandatory in specialist settings.",
47    "onset": "Hours to days"
9    },
48    {
0      "drugs": ["amiodarone", "azithromycin"], "severity": "MAJOR",
    -   "mechanism": "Additive QTc prolongation (both prolong cardiac repolarisation via IKr blockade).",

```

```

-         "effect": "Torsades de pointes.",
48         "recommendation": "Avoid. Use doxycycline or co-amoxiclav as alternative to azithromycin.",
1         "onset": "Hours to days"
-     },
48     {
2         "drugs": ["amiodarone", "clarithromycin"], "severity": "MAJOR",
-         "mechanism": "Additive QTc prolongation; clarithromycin also inhibits CYP3A4 (amiodarone metabolism), raising
48 amiodarone levels.",
3         "effect": "Torsades de pointes.",
-         "recommendation": "Avoid. Use alternative antibiotic.",
48         "onset": "Hours to days"
4     },
-     {
48     {
5         "drugs": ["amiodarone", "haloperidol"], "severity": "MAJOR",
-         "mechanism": "Additive QTc prolongation.",
48         "effect": "Torsades de pointes.",
6         "recommendation": "Avoid. If necessary, baseline ECG and regular QTc monitoring.",
-         "onset": "Hours to days"
48     },
7     {
-         "drugs": ["amiodarone", "ciprofloxacin"], "severity": "MAJOR",
48         "mechanism": "Additive QTc prolongation (fluoroquinolone IKr blockade).",
8         "effect": "Torsades de pointes.",
-         "recommendation": "Avoid. Use alternative antibiotic.",
48         "onset": "Hours to days"
9     },
-     {
49     {
0         "drugs": ["haloperidol", "azithromycin"], "severity": "MAJOR",
-         "mechanism": "Additive QTc prolongation.",
49         "effect": "Torsades de pointes.",
1         "recommendation": "Avoid. Use alternative antibiotic.",
-         "onset": "Hours to days"
49     },
1     {
49     {
2         "drugs": ["simvastatin", "amiodarone"], "severity": "MAJOR",
-         "mechanism": "Amiodarone inhibits CYP3A4 (simvastatin metabolism); simvastatin has concentration-dependent
- myopathy.",
49         "effect": "Myopathy; rhabdomyolysis.",
3         "recommendation": "Simvastatin dose must not exceed 20 mg/day with amiodarone. Switch to pravastatin or
- rosuvastatin (less CYP3A4-dependent).",
49         "onset": "Weeks to months"
4     },
-     {
49     {
5         "drugs": ["simvastatin", "clarithromycin"], "severity": "MAJOR",
-         "mechanism": "Clarithromycin inhibits CYP3A4, markedly raising simvastatin acid levels.",
49         "effect": "Myopathy; rhabdomyolysis.",
6         "recommendation": "Suspend simvastatin for the duration of the clarithromycin course. Restart after completion.",
-         "onset": "Days"
49     },
7     {
-         "drugs": ["atorvastatin", "clarithromycin"], "severity": "MAJOR",
49         "mechanism": "Clarithromycin inhibits CYP3A4 (atorvastatin metabolism).",
8         "effect": "Elevated atorvastatin levels; myopathy risk.",
-         "recommendation": "Suspend atorvastatin during clarithromycin course or switch to pravastatin/rosuvastatin.",
49         "onset": "Days"
9     },
-     {
50     {
0         "drugs": ["simvastatin", "ciclosporin"], "severity": "MAJOR",
-         "mechanism": "Ciclosporin inhibits both OATP1B1 transporter (hepatic uptake) and CYP3A4, markedly increasing
- simvastatin bioavailability.",
50         "effect": "Severe myopathy; rhabdomyolysis.",
1         "recommendation": "Avoid simvastatin in patients on ciclosporin. Use pravastatin (lower OATP1B1 dependence) at low
- doses.",
50         "onset": "Weeks"
2     },
-     {
50     {
3         "drugs": ["simvastatin", "fluconazole"], "severity": "MAJOR",
-         "mechanism": "Fluconazole inhibits CYP3A4, raising simvastatin levels markedly.",
50         "effect": "Myopathy; rhabdomyolysis.",
4         "recommendation": "Suspend simvastatin during fluconazole course.",
-         "onset": "Days"
50     },
5     {
-         "drugs": ["spironolactone", "ramipril"], "severity": "MAJOR",
50         "mechanism": "Both raise serum potassium: aldosterone antagonist + reduced aldosterone synthesis via ACE
- inhibition.",
50         "effect": "Hyperkalaemia: cardiac arrhythmias, cardiac arrest.",
6         "recommendation": "Monitor potassium at baseline, 1-2 weeks, then monthly. Acceptable in heart failure if eGFR >30
- and K+ <5.0 mmol/L at baseline.",

```

```

50     "onset": "Days to weeks"
7     },
-     {
50         "drugs": ["spironolactone", "lisinopril"], "severity": "MAJOR",
8         "mechanism": "Dual hyperkalaemia (aldosterone antagonist + ACE inhibitor).",
-         "effect": "Hyperkalaemia.",
50         "recommendation": "Monitor potassium at 1-2 weeks and monthly.",
9         "onset": "Days to weeks"
-     },
51     {
0         "drugs": ["allopurinol", "azathioprine"], "severity": "MAJOR",
-         "mechanism": "Allopurinol inhibits xanthine oxidase, the enzyme responsible for inactivating azathioprine's active
51 metabolite (6-mercaptopurine), causing accumulation.",
1         "effect": "Severe bone marrow suppression: pancytopenia, agranulocytosis.",
-         "recommendation": "Avoid if at all possible. If essential, reduce azathioprine dose by 75% and monitor FBC very
51 frequently.",
2         "onset": "Days to weeks"
-     },
51     {
3         "drugs": ["valproate", "lamotrigine"], "severity": "MAJOR",
-         "mechanism": "Valproate inhibits glucuronidation of lamotrigine, approximately doubling plasma lamotrigine
51 levels.",
4         "effect": "Lamotrigine toxicity (ataxia, diplopia, dizziness); elevated Stevens-Johnson syndrome risk with rapid
51 dose escalation.",
-         "recommendation": "Halve the standard starting lamotrigine dose and titrate at half the usual rate when adding to
5 valproate.",
-         "onset": "Days to weeks"
51     },
6     {
-         "drugs": ["carbamazepine", "lamotrigine"], "severity": "MAJOR",
51         "mechanism": "Carbamazepine induces lamotrigine glucuronidation, reducing lamotrigine levels 40-50%.",
7         "effect": "Subtherapeutic lamotrigine levels; worsened seizure control.",
-         "recommendation":
51 "Higher lamotrigine doses required when co-prescribed with carbamazepine. Monitor seizure control carefully.",
8         "onset": "Weeks"
-     },
51     {
9         "drugs": ["ramipril", "ibuprofen"], "severity": "MAJOR",
-         "mechanism": "NSAIDs reduce renal prostaglandin synthesis (reduces GFR) and antagonise ACE inhibitor
52 antihypertensive effect. With a concurrent diuretic, this constitutes the 'triple whammy' - a leading cause of drug-induced
0 AKI.",
-         "effect": "Acute kidney injury; loss of blood pressure control.",
52         "recommendation": "Avoid. Use paracetamol. If NSAID essential, stop ACE inhibitor temporarily and monitor renal
1 function.",
-         "onset": "Days"
52     },
2     {
-         "drugs": ["lisinopril", "ibuprofen"], "severity": "MAJOR",
52         "mechanism": "NSAID reduces GFR and antagonises ACE inhibitor. With diuretic = triple whammy.",
3         "effect": "Acute kidney injury.",
-         "recommendation": "Avoid. Use paracetamol.",
52         "onset": "Days"
4     },
-     {
52     {
5         "drugs": ["theophylline", "ciprofloxacin"], "severity": "MAJOR",
-         "mechanism": "Ciprofloxacin inhibits CYP1A2, the primary enzyme metabolising theophylline.",
52         "effect": "Theophylline toxicity: tachycardia, nausea, vomiting, seizures, arrhythmias.",
-         "recommendation": "Reduce theophylline dose by 30-50% during ciprofloxacin. Monitor levels. Use alternative
52 antibiotic if possible.",
6         "onset": "Days"
-     },
52     {
7         "drugs": ["theophylline", "clarithromycin"], "severity": "MAJOR",
-         "mechanism": "Clarithromycin inhibits CYP1A2 and CYP3A4 (theophylline metabolism).",
52         "effect": "Theophylline toxicity.",
8         "recommendation": "Monitor theophylline levels closely. Consider alternative antibiotic.",
-         "onset": "Days"
52     },
9     {
-         "drugs": ["apixaban", "aspirin"], "severity": "MAJOR",
53         "mechanism": "Additive anticoagulant + antiplatelet bleeding risk.",
0         "effect": "Major bleeding including intracranial haemorrhage.",
-         "recommendation": "Avoid unless dual antithrombotic therapy is specifically indicated (ACS/stent + AF). Add PPI.",
53         "onset": "Immediate"
1     },
-     {
53     {
2         "drugs": ["rivaroxaban", "aspirin"], "severity": "MAJOR",
-         "mechanism": "Additive anticoagulant + antiplatelet bleeding risk.",
53         "effect": "Major bleeding.",

```

```

3      "recommendation": "Avoid unless clinically indicated. Add PPI.",
      "onset": "Immediate"
53  },
4  {
      "drugs": ["apixaban", "ibuprofen"], "severity": "MAJOR",
53      "mechanism": "NSAIDs: antiplatelet + GI mucosal damage; additive with DOAC anticoagulation.",
5      "effect": "GI haemorrhage.",
      "recommendation": "Avoid NSAIDs in patients on DOACs. Use paracetamol.",
53      "onset": "Days"
6  },
      {
53      "drugs": ["rivaroxaban", "ibuprofen"], "severity": "MAJOR",
7      "mechanism": "Additive bleeding risk (DOAC + NSAID).",
      "effect": "GI haemorrhage.",
53      "recommendation": "Avoid NSAIDs with DOACs.",
8      "onset": "Days"
      },
53  {
9      "drugs": ["colchicine", "clarithromycin"], "severity": "MAJOR",
      "mechanism": "Clarithromycin inhibits both P-glycoprotein (colchicine efflux) and CYP3A4 (colchicine metabolism),
54 dramatically raising colchicine plasma levels.",
0      "effect": "Colchicine toxicity: myopathy, bone marrow suppression, multi-organ failure. Fatal cases reported.",
      "recommendation": "Contraindicated in renal/hepatic impairment. In normal renal/hepatic function, use the lowest
54 colchicine dose for the shortest duration.",
1      "onset": "Days"
      },
54  {
2      "drugs": ["colchicine", "ciclosporin"], "severity": "MAJOR",
      "mechanism": "Ciclosporin inhibits P-glycoprotein and CYP3A4 (colchicine clearance).",
54      "effect": "Colchicine toxicity.",
3      "recommendation": "Avoid. Use alternative gout therapy.",
      "onset": "Days"
54  },
4  {
      "drugs": ["ciclosporin", "ibuprofen"], "severity": "MAJOR",
54      "mechanism": "NSAIDs reduce renal prostaglandins; ciclosporin has direct nephrotoxicity – combination markedly
5 increases AKI risk.",
      "effect": "Acute kidney injury.",
54      "recommendation": "Avoid. Use paracetamol.",
6      "onset": "Days"
      },
54  {
7      "drugs": ["tacrolimus", "ibuprofen"], "severity": "MAJOR",
      "mechanism": "Additive nephrotoxicity (calcineurin inhibitor + NSAID).",
54      "effect": "Acute kidney injury.",
8      "recommendation": "Avoid. Use paracetamol.",
      "onset": "Days"
54  },
9  # — MODERATE —————
      {
55      "drugs": ["clopidogrel", "omeprazole"], "severity": "MODERATE",
0      "mechanism": "Omeprazole inhibits CYP2C19, which activates clopidogrel (prodrug → active thiol metabolite).",
      "effect": "Reduced antiplatelet activity of clopidogrel; potential increased cardiovascular events.",
55      "recommendation": "Use pantoprazole or esomeprazole (less CYP2C19 inhibition) instead of omeprazole as the PPI.",
1      "onset": "Days"
      },
55  {
2      "drugs": ["quetiapine", "azithromycin"], "severity": "MODERATE",
      "mechanism": "Additive QTc prolongation.",
55      "effect": "Torsades de pointes risk.",
3      "recommendation": "Baseline ECG. Monitor QTc. Use alternative antibiotic if feasible.",
      "onset": "Hours to days"
55  },
4  {
      "drugs": ["gabapentin", "morphine"], "severity": "MODERATE",
55      "mechanism": "Additive CNS and respiratory depression.",
5      "effect": "Excessive sedation, respiratory depression.",
      "recommendation": "Use lowest effective doses. Monitor for respiratory depression. Avoid in patients with COPD.",
55      "onset": "Hours"
6  },
      {
55      "drugs": ["pregabalin", "oxycodone"], "severity": "MODERATE",
7      "mechanism": "Additive CNS depression (gabapentinoid + opioid).",
      "effect": "Sedation, cognitive impairment, respiratory depression.",
55      "recommendation": "Lowest effective doses. Counsel on driving. Monitor closely.",
8      "onset": "Hours"
      },
55  {
9      "drugs": ["pregabalin", "morphine"], "severity": "MODERATE",

```

```

-         "mechanism": "Additive CNS and respiratory depression.",
56         "effect": "Respiratory depression.",
0         "recommendation": "Start low. Monitor.",
-         "onset": "Hours"
56     },
1     {
-         "drugs": ["furosemide", "ibuprofen"], "severity": "MODERATE",
56         "mechanism": "NSAIDs antagonise the renal prostaglandin-dependent mechanism of loop diuretics.",
2         "effect": "Reduced diuretic efficacy; fluid retention; AKI risk.",
-         "recommendation": "Avoid NSAIDs in patients requiring diuretic therapy. Use paracetamol.",
56         "onset": "Days"
3     },
-     {
56         "drugs": ["sertraline", "ibuprofen"], "severity": "MODERATE",
4         "mechanism":
-         "SSRIs deplete platelet serotonin (reducing platelet aggregation); NSAIDs add independent antiplatelet and mucosal
56         effects.",
5         "effect": "3–15× increased risk of upper GI bleeding.",
-         "recommendation": "Add PPI cover if both drugs are necessary.",
56         "onset": "Days to weeks"
6     },
-     {
56         "drugs": ["fluoxetine", "ibuprofen"], "severity": "MODERATE",
7         "mechanism": "SSRI platelet depletion + NSAID antiplatelet/mucosal effects.",
-         "effect": "Increased GI bleeding risk.",
56         "recommendation": "Add PPI cover.",
8         "onset": "Days to weeks"
-     },
56     {
9         "drugs": ["simvastatin", "amlodipine"], "severity": "MODERATE",
-         "mechanism": "Amlodipine weakly inhibits CYP3A4, modestly increasing simvastatin exposure.",
57         "effect": "Slightly increased myopathy risk.",
-         "recommendation": "Simvastatin dose should not exceed 20 mg/day when co-prescribed with amlodipine.",
57         "onset": "Weeks"
1     },
-     {
57         "drugs": ["levothyroxine", "warfarin"], "severity": "MODERATE",
-         "mechanism": "Thyroid hormones increase catabolism of vitamin K-dependent clotting factors.",
57         "effect": "Elevated INR when levothyroxine dose is increased or a hypothyroid patient achieves euthyroidism.",
2         "recommendation": "Monitor INR whenever levothyroxine dose changes.",
57         "onset": "Weeks"
3     },
-     {
57         "drugs": ["carbamazepine", "valproate"], "severity": "MODERATE",
4         "mechanism": "Carbamazepine induces valproate metabolism; valproate inhibits carbamazepine epoxide hydrolase,
-         raising the toxic carbamazepine-10,11-epoxide metabolite.",
57         "effect": "Reduced valproate levels; carbamazepine toxicity (diplopia, ataxia, dizziness) from raised epoxide.",
5         "recommendation": "Monitor both drug levels. Specialist epilepsy supervision recommended.",
-         "onset": "Weeks"
57     },
6     {
-         "drugs": ["metformin", "ibuprofen"], "severity": "MODERATE",
57         "mechanism": "NSAIDs reduce GFR; metformin accumulates in renal impairment with lactic acidosis risk.",
7         "effect": "Metformin accumulation and risk of lactic acidosis, particularly in CKD.",
-         "recommendation": "Use paracetamol instead. Monitor renal function if NSAID unavoidable.",
57         "onset": "Days"
8     },
-     {
57         "drugs": ["enoxaparin", "aspirin"], "severity": "MODERATE",
9         "mechanism": "Additive anticoagulant + antiplatelet bleeding risk.",
-         "effect": "Increased bleeding.",
58         "recommendation": "Often co-prescribed intentionally in ACS – acceptable with clear clinical indication and PPI
0         cover.",
-         "onset": "Hours"
58     },
1     {
-         "drugs": ["ramipril", "furosemide"], "severity": "MODERATE",
58         "mechanism": "Furosemide-induced volume depletion sensitises patients to first-dose ACE inhibitor hypotension.",
2         "effect": "Symptomatic first-dose hypotension.",
-         "recommendation": "Start ACE inhibitor at lowest dose. Consider stopping furosemide 24h before initiating ACE
58         inhibitor in volume-depleted patients.",
3         "onset": "Hours (first dose)"
-     },
58 ]
4
-
58 def _check_drug_interactions(drug_list: list[str]) -> dict:
5     """Check a list of drug name strings for known clinically significant interactions."""
-

```

```

58 # Build symmetric lookup: frozenset({drug_a, drug_b}) → interaction
6 ix_lookup: dict[frozenset, dict] = {}
7 for ix in DRUG_INTERACTIONS:
58     key = frozenset(ix["drugs"])
7     if key not in ix_lookup:
58         ix_lookup[key] = ix
58
8 # Normalise each drug name: lowercase, strip dose/frequency, resolve aliases
_med_lower = [m.lower() for m in _NLP_MEDICATIONS]
58
9 def _normalise(raw: str) -> str | None:
    name = raw.strip().lower()
    # Strip trailing dose / frequency info
0    name = re.sub(r'\s+\d[\d.]*\s*(mg|mcg|g|ml|mL|units?|IU|%)\S*', '', name)
    name = re.sub(
59        r'\s+(od|bd|tds|qds|prn|nocte|stat|once\s+daily|twice\s+daily|'
1        r'three\s+times|four\s+times)\b.*', '', name, flags=re.I
    )
59    name = name.strip()
2    # Check aliases
    name = _DRUG_ALIASES.get(name, name)
59    # Exact match
3    if name in _med_lower:
        return name
59    # Partial match: name is a substring of a known drug (handles brand-ish substrings)
4    for med in _med_lower:
        if name and (name in med or med in name) and len(name) >= 4:
59            return med
5    return None
59
6 recognised: list[str] = []
unrecognised: list[str] = []
7 for raw in drug_list:
59     norm = _normalise(raw)
7     if norm and norm not in recognised:
59         recognised.append(norm)
59     elif not norm:
8         clean = raw.strip()
59         if clean and clean not in unrecognised:
9             unrecognised.append(clean)
59
60 # Check all unique pairs
interactions_found: list[dict] = []
0 sev_counts = {"CONTRAINDICATED": 0, "MAJOR": 0, "MODERATE": 0, "MINOR": 0}
60
61 for i in range(len(recognised)):
    for j in range(i + 1, len(recognised)):
60         key = frozenset([recognised[i], recognised[j]])
2         if key in ix_lookup:
59             ix = ix_lookup[key]
60             interactions_found.append({
3                 **ix,
60                 "drug_a": recognised[i],
4                 "drug_b": recognised[j],
60             })
            sev = ix["severity"]
            if sev in sev_counts:
60                 sev_counts[sev] += 1
5
60 # Sort by severity
_sev_order = {"CONTRAINDICATED": 0, "MAJOR": 1, "MODERATE": 2, "MINOR": 3}
6 interactions_found.sort(key=lambda x: _sev_order.get(x["severity"], 4))
60
61 return {
7     "drugs_detected": recognised,
    "drugs_unrecognised": unrecognised,
60    "interactions": interactions_found,
8    "severity_counts": sev_counts,
60    "total": len(interactions_found),
9    }
61
62 @app.route("/drug-checker")
0 def drug_checker():
    return render_template("drug_checker.html")
61
62 @app.route("/drug-checker/run", methods=["POST"])
61 def drug_checker_run():

```

```

2     try:
3         data = request.get_json(force=True) or {}
4         text = data.get("text", "").strip()
5         if not text:
6             return jsonify({"ok": False, "error": "No medications provided"}), 400
7
8         # Extract medications from text using NLP extractor
9         nlp_result = _run_nlp_extractor(text)
10        drug_list = [e["text"] for e in nlp_result["entities"] if e["type"] == "MEDICATION"]
11
12        # Also parse line-by-line (handles plain lists and bullet-point lists)
13        lines = [l.strip() for l in text.split("\n") if l.strip()]
14        for line in lines:
15            # Strip bullets / numbers: "- warfarin 5mg" → "warfarin 5mg"
16            line = re.sub(r'^[-\s*\d\.\s]*', '', line).strip()
17            if line and line not in drug_list:
18                drug_list.append(line)
19
20        result = _check_drug_interactions(drug_list)
21        return jsonify({"ok": True, **result})
22    except Exception as exc:
23        return jsonify({"ok": False, "error": str(exc)}), 500
24
25# — LLM Agent Trace Visualiser —
26
27def _parse_agent_trace(raw: dict | list) -> dict:
28    """Normalise Claude API, OpenAI, or generic messages-array traces into a unified structure."""
29
30    # Detect format and extract messages list
31    if isinstance(raw, list):
32        msgs = raw
33        model = "unknown"
34    elif isinstance(raw, dict):
35        if "messages" in raw:
36            msgs = raw["messages"]
37            model = raw.get("model", raw.get("id", "unknown"))
38        elif "role" in raw: # single Anthropic response object
39            msgs = [raw]
40            model = raw.get("model", "unknown")
41        elif "choices" in raw: # OpenAI Chat Completion response
42            msgs = [c.get("message", {}) for c in raw.get("choices", [])]
43            model = raw.get("model", "unknown")
44        else:
45            raise ValueError("Unrecognised trace format – expected messages array, Anthropic response, or OpenAI
46response.")
47    else:
48        raise ValueError("Trace must be a JSON object or array.")
49
50    steps = []
51    total_input = 0
52    total_output = 0
53    tool_counts: dict[str, int] = {}
54
55    for i, msg in enumerate(msgs):
56        role = msg.get("role", "unknown")
57        content = msg.get("content", "")
58        usage = msg.get("usage", {})
59
60        # Token counts – Anthropic uses input_tokens/output_tokens;
61        # OpenAI uses prompt_tokens/completion_tokens at the top level
62        inp = usage.get("input_tokens", usage.get("prompt_tokens", 0))
63        out = usage.get("output_tokens", usage.get("completion_tokens", 0))
64        total_input += inp
65        total_output += out
66
67        blocks = []
68
69        # Normalise content to a list of typed blocks
70        if isinstance(content, str) and content:
71            blocks.append({"type": "text", "text": content})
72        elif isinstance(content, list):
73            for block in content:
74                if not isinstance(block, dict):
75                    continue
76                btype = block.get("type", "text")
77
78                if btype == "text":
79                    blocks.append({"type": "text", "text": block.get("text", "")})

```

```

63         elif btype == "thinking":
64             blocks.append({"type": "thinking", "text": block.get("thinking", "")})
65
66         elif btype == "tool_use":
67             name = block.get("name", "")
68             tool_counts[name] = tool_counts.get(name, 0) + 1
69             blocks.append({
70                 "type": "tool_use",
71                 "id": block.get("id", ""),
72                 "name": name,
73                 "input": block.get("input", {}),
74             })
75
76         elif btype == "tool_result":
77             rc = block.get("content", "")
78             if isinstance(rc, list):
79                 rc = "\n".join(
80                     c.get("text", "") for c in rc
81                     if isinstance(c, dict) and c.get("type") == "text"
82                 )
83             blocks.append({
84                 "type": "tool_result",
85                 "tool_use_id": block.get("tool_use_id", ""),
86                 "content": rc,
87                 "is_error": block.get("is_error", False),
88             })
89
90         # Handle OpenAI tool_calls field (parallel to content)
91         for tc in msg.get("tool_calls", []):
92             fn = tc.get("function", {})
93             args = fn.get("arguments", {})
94             if isinstance(args, str):
95                 try:
96                     args = json.loads(args)
97                 except Exception:
98                     args = {"raw": args}
99             name = fn.get("name", "")
100             tool_counts[name] = tool_counts.get(name, 0) + 1
101             blocks.append({
102                 "type": "tool_use",
103                 "id": tc.get("id", ""),
104                 "name": name,
105                 "input": args,
106             })
107
108         # OpenAI tool response messages (role == "tool")
109         if role == "tool":
110             blocks.append({
111                 "type": "tool_result",
112                 "tool_use_id": msg.get("tool_call_id", ""),
113                 "content": content if isinstance(content, str) else str(content),
114                 "is_error": False,
115             })
116
117         steps.append({
118             "index": i,
119             "role": role,
120             "blocks": blocks,
121             "input_tokens": inp,
122             "output_tokens": out,
123         })
124
125     return {
126         "steps": steps,
127         "model": model,
128         "total_input_tokens": total_input,
129         "total_output_tokens": total_output,
130         "total_tokens": total_input + total_output,
131         "total_steps": len(steps),
132         "tool_counts": tool_counts,
133     }

```

```

3 @app.route("/agent-trace")
4 def agent_trace():
5     return render_template("agent_trace.html")

```

```

66 @app.route("/agent-trace/parse", methods=["POST"])
5 def agent_trace_parse():
    try:
66         data = request.get_json(force=True) or {}
6         raw = data.get("trace")
        if raw is None:
66             return jsonify({"ok": False, "error": "No trace provided"}), 400
7         result = _parse_agent_trace(raw)
        return jsonify({"ok": True, **result})
66     except Exception as exc:
8         return jsonify({"ok": False, "error": str(exc)}), 500

66
9 @app.route("/binaural")
def binaural():
67     return render_template("binaural.html")
0

67 @app.route("/about")
1 def about():
    return render_template("about.html")
67
2
@app.route("/contact", methods=["GET", "POST"])
67 def contact():
3     if request.method == "GET":
        return render_template("contact.html")
67
4     name = request.form.get("name", "").strip()
    email = request.form.get("email", "").strip()
67     subject = request.form.get("subject", "Message from drnealaggarwal.info").strip()
5     message = request.form.get("message", "").strip()

67     if not all([name, email, message]):
6         flash("Please fill in all required fields.", "error")
        return redirect(url_for("contact"))
67
7     try:
        gmail_user = os.environ.get("GMAIL_USER", "dr.neal.aggarwal@gmail.com")
        gmail_pass = os.environ.get("GMAIL_APP_PASS", "")

        body = (
67             f"From: {name} <{email}>\n"
9             f"Subject: {subject}\n"
            f"{'-'*48}\n\n"
68             f"{message}"
0         )

68         msg = MIME multipart()
1         msg["From"] = gmail_user
        msg["To"] = "dr.neal.aggarwal@gmail.com"
68         msg["Reply-To"] = email
2         msg["Subject"] = f"[drnealaggarwal.info] {subject}"
        msg.attach(MIMEText(body, "plain"))
68
3         if gmail_pass:
            with smtplib.SMTP_SSL("smtp.gmail.com", 465) as smtp:
68                 smtp.login(gmail_user, gmail_pass)
4                 smtp.send_message(msg)

68         flash("Message sent – I'll be in touch.", "success")
5     except Exception:
        flash("Something went wrong. Please email me directly at dr.neal.aggarwal@gmail.com.", "error")
68
6     return redirect(url_for("contact"))

68
7 @app.route("/sw.js")
def service_worker():
68     from flask import send_from_directory
8     return send_from_directory(BASE / "static", "sw.js", mimetype="application/javascript")

68
9 @app.errorhandler(404)
def not_found(e):
69     return render_template("404.html"), 404
0

```

```
1
_
69
2
_
69
3
_
69
4
_
69
5
_
69
6
_
69
7
_
69
8
_
69
9
_
70
0
_
70
1
_
70
2
_
70
3
_
70
4
if __name__ == "__main__":
    app.run(debug=True, host="0.0.0.0", port=5001)
_
70
5
_
70
6
_
70
7
_
70
8
_
70
9
_
71
0
_
71
1
_
71
2
_
71
3
_
71
4
_
71
5
_
71
6
_
71
7
```

71
8
71
9
72
0
72
1
72
2
72
3
72
4
72
5
72
6
72
7
72
8
72
9
73
0
73
1
73
2
73
3
73
4
73
5
73
6
73
7
73
8
73
9
74
0
74
1
74
2
74
3
-

74
4
74
5
74
6
74
7
74
8
74
9
75
0
75
1
75
2
75
3
75
4
75
5
75
6
75
7
75
8
75
9
76
0
76
1
76
2
76
3
76
4
76
5
76
6
76
7
76
8
76
9
77

0
77
1
77
2
77
3
77
4
77
5
77
6
77
7
77
8
77
9
78
0
78
1
78
2
78
3
78
4
78
5
78
6
78
7
78
8
78
9
79
0
79
1
79
2
79
3
79
4
79
5
79
6

79
7
79
8
79
9
80
0
80
1
80
2
80
3
80
4
80
5
80
6
80
7
80
8
80
9
81
0
81
1
81
2
81
3
81
4
81
5
81
6
81
7
81
8
81
9
82
0
82
1
82
2
-

82
3
-
82
4
-
82
5
-
82
6
-
82
7
-
82
8
-
82
9
-
83
0
-
83
1
-
83
2
-
83
3
-
83
4
-
83
5
-
83
6
-
83
7
-
83
8
-
83
9
-
84
0
-
84
1
-
84
2
-
84
3
-
84
4
-
84
5
-
84
6
-
84
7
-
84
8
-
84

9
-
85
0
-
85
1
-
85
2
-
85
3
-
85
4
-
85
5
-
85
6
-
85
7
-
85
8
-
85
9
-
86
0
-
86
1
-
86
2
-
86
3
-
86
4
-
86
5
-
86
6
-
86
7
-
86
8
-
86
9
-
87
0
-
87
1
-
87
2
-
87
3
-
87
4
-
87
5

87
6
87
7
87
8
87
9
88
0
88
1
88
2
88
3
88
4
88
5
88
6
88
7
88
8
88
9
89
0
89
1
89
2
89
3
89
4
89
5
89
6
89
7
89
8
89
9
90
0
90
1
-

90
2
-
90
3
-
90
4
-
90
5
-
90
6
-
90
7
-
90
8
-
90
9
-
91
0
-
91
1
-
91
2
-
91
3
-
91
4
-
91
5
-
91
6
-
91
7
-
91
8
-
91
9
-
92
0
-
92
1
-
92
2
-
92
3
-
92
4
-
92
5
-
92
6
-
92
7
-
92

8
-
92
9
-
93
0
-
93
1
-
93
2
-
93
3
-
93
4
-
93
5
-
93
6
-
93
7
-
93
8
-
93
9
-
94
0
-
94
1
-
94
2
-
94
3
-
94
4
-
94
5
-
94
6
-
94
7
-
94
8
-
94
9
-
95
0
-
95
1
-
95
2
-
95
3
-
95
4

[95](#)
5
[95](#)
6
[95](#)
7
[95](#)
8
[95](#)
9
[96](#)
0
[96](#)
1
[96](#)
2
[96](#)
3
[96](#)
4
[96](#)
5
[96](#)
6
[96](#)
7
[96](#)
8
[96](#)
9
[97](#)
0
[97](#)
1
[97](#)
2
[97](#)
3
[97](#)
4
[97](#)
5
[97](#)
6
[97](#)
7
[97](#)
8
[97](#)
9
[98](#)
0
—

98
1
-
98
2
-
98
3
-
98
4
-
98
5
-
98
6
-
98
7
-
98
8
-
98
9
-
99
0
-
99
1
-
99
2
-
99
3
-
99
4
-
99
5
-
99
6
-
99
7
-
99
8
-
99
9
100
0
100
1
100
2
100
3
100
4
100
5
100
6
100
7
100
8
100
9
101
0
101

1
101
2
101
3
101
4
101
5
101
6
101
7
101
8
101
9
102
0
102
1
102
2
102
3
102
4
102
5
102
6
102
7
102
8
102
9
103
0
103
1
103
2
103
3
103
4
103
5
103
6
103
7
103
8
103
9
104
0
104
1
104
2
104
3
104
4
104
5
104
6
104
7
104
8
104
9
105
0

105
1
105
2
105
3
105
4
105
5
105
6
105
7
105
8
105
9
106
0
106
1
106
2
106
3
106
4
106
5
106
6
106
7
106
8
106
9
107
0
107
1
107
2
107
3
107
4
107
5
107
6
107
7
107
8
107
9
108
0
108
1
108
2
108
3
108
4
108
5
108
6
108
7
108
8
108
9
109

0
109
1
109
2
109
3
109
4
109
5
109
6
109
7
109
8
109
9
110
0
110
1
110
2
110
3
110
4
110
5
110
6
110
7
110
8
110
9
111
0
111
1
111
2
111
3
111
4
111
5
111
6
111
7
111
8
111
9
112
0
112
1
112
2
112
3
112
4
112
5
112
6
112
7
112
8
112
9

[113](#)
[0](#)
[113](#)
[1](#)
[113](#)
[2](#)
[113](#)
[3](#)
[113](#)
[4](#)
[113](#)
[5](#)
[113](#)
[6](#)
[113](#)
[7](#)
[113](#)
[8](#)
[113](#)
[9](#)
[114](#)
[0](#)
[114](#)
[1](#)
[114](#)
[2](#)
[114](#)
[3](#)
[114](#)
[4](#)
[114](#)
[5](#)
[114](#)
[6](#)
[114](#)
[7](#)
[114](#)
[8](#)
[114](#)
[9](#)
[115](#)
[0](#)
[115](#)
[1](#)
[115](#)
[2](#)
[115](#)
[3](#)
[115](#)
[4](#)
[115](#)
[5](#)
[115](#)
[6](#)
[115](#)
[7](#)
[115](#)
[8](#)
[115](#)
[9](#)
[116](#)
[0](#)
[116](#)
[1](#)
[116](#)
[2](#)
[116](#)
[3](#)
[116](#)
[4](#)
[116](#)
[5](#)
[116](#)
[6](#)
[116](#)
[7](#)
[116](#)
[8](#)
[116](#)

9
117
0
117
1
117
2
117
3
117
4
117
5
117
6
117
7
117
8
117
9
118
0
118
1
118
2
118
3
118
4
118
5
118
6
118
7
118
8
118
9
119
0
119
1
119
2
119
3
119
4
119
5
119
6
119
7
119
8
119
9
120
0
120
1
120
2
120
3
120
4
120
5
120
6
120
7
120
8

120
9
121
0
121
1
121
2
121
3
121
4
121
5
121
6
121
7
121
8
121
9
122
0
122
1
122
2
122
3
122
4
122
5
122
6
122
7
122
8
122
9
123
0
123
1
123
2
123
3
123
4
123
5
123
6
123
7
123
8
123
9
124
0
124
1
124
2
124
3
124
4
124
5
124
6
124
7
124

8
124
9
125
0
125
1
125
2
125
3
125
4
125
5
125
6
125
7
125
8
125
9
126
0
126
1
126
2
126
3
126
4
126
5
126
6
126
7
126
8
126
9
127
0
127
1
127
2
127
3
127
4
127
5
127
6
127
7
127
8
127
9
128
0
128
1
128
2
128
3
128
4
128
5
128
6
128
7

[128](#)
8
[128](#)
9
[129](#)
0
[129](#)
1
[129](#)
2
[129](#)
3
[129](#)
4
[129](#)
5
[129](#)
6
[129](#)
7
[129](#)
8
[129](#)
9
[130](#)
0
[130](#)
1
[130](#)
2
[130](#)
3
[130](#)
4
[130](#)
5
[130](#)
6
[130](#)
7
[130](#)
8
[130](#)
9
[131](#)
0
[131](#)
1
[131](#)
2
[131](#)
3
[131](#)
4
[131](#)
5
[131](#)
6
[131](#)
7
[131](#)
8
[131](#)
9
[132](#)
0
[132](#)
1
[132](#)
2
[132](#)
3
[132](#)
4
[132](#)
5
[132](#)
6
[132](#)

7
132
8
132
9
133
0
133
1
133
2
133
3
133
4
133
5
133
6
133
7
133
8
133
9
134
0
134
1
134
2
134
3
134
4
134
5
134
6
134
7
134
8
134
9
135
0
135
1
135
2
135
3
135
4
135
5
135
6
135
7
135
8
135
9
136
0
136
1
136
2
136
3
136
4
136
5
136
6

[136](#)
7
[136](#)
8
[136](#)
9
[137](#)
0
[137](#)
1
[137](#)
2
[137](#)
3
[137](#)
4
[137](#)
5
[137](#)
6
[137](#)
7
[137](#)
8
[137](#)
9
[138](#)
0
[138](#)
1
[138](#)
2
[138](#)
3
[138](#)
4
[138](#)
5
[138](#)
6
[138](#)
7
[138](#)
8
[138](#)
9
[139](#)
0
[139](#)
1
[139](#)
2
[139](#)
3
[139](#)
4
[139](#)
5
[139](#)
6
[139](#)
7
[139](#)
8
[139](#)
9
[140](#)
0
[140](#)
1
[140](#)
2
[140](#)
3
[140](#)
4
[140](#)
5
[140](#)

6
[140](#)
7
[140](#)
8
[140](#)
9
[141](#)
0
[141](#)
1
[141](#)
2
[141](#)
3
[141](#)
4
[141](#)
5
[141](#)
6
[141](#)
7
[141](#)
8
[141](#)
9
[142](#)
0
[142](#)
1
[142](#)
2
[142](#)
3
[142](#)
4
[142](#)
5
[142](#)
6
[142](#)
7
[142](#)
8
[142](#)
9
[143](#)
0
[143](#)
1
[143](#)
2
[143](#)
3
[143](#)
4
[143](#)
5
[143](#)
6
[143](#)
7
[143](#)
8
[143](#)
9
[144](#)
0
[144](#)
1
[144](#)
2
[144](#)
3
[144](#)
4
[144](#)
5

[144](#)
6
[144](#)
7
[144](#)
8
[144](#)
9
[145](#)
0
[145](#)
1
[145](#)
2
[145](#)
3
[145](#)
4
[145](#)
5
[145](#)
6
[145](#)
7
[145](#)
8
[145](#)
9
[146](#)
0
[146](#)
1
[146](#)
2
[146](#)
3
[146](#)
4
[146](#)
5
[146](#)
6
[146](#)
7
[146](#)
8
[146](#)
9
[147](#)
0
[147](#)
1
[147](#)
2
[147](#)
3
[147](#)
4
[147](#)
5
[147](#)
6
[147](#)
7
[147](#)
8
[147](#)
9
[148](#)
0
[148](#)
1
[148](#)
2
[148](#)
3
[148](#)
4
[148](#)

5
148
6
148
7
148
8
148
9
149
0
149
1
149
2
149
3
149
4
149
5
149
6
149
7
149
8
149
9
150
0
150
1
150
2
150
3
150
4
150
5
150
6
150
7
150
8
150
9
151
0
151
1
151
2
151
3
151
4
151
5
151
6
151
7
151
8
151
9
152
0
152
1
152
2
152
3
152
4

[152](#)
5
[152](#)
6
[152](#)
7
[152](#)
8
[152](#)
9
[153](#)
0
[153](#)
1
[153](#)
2
[153](#)
3
[153](#)
4
[153](#)
5
[153](#)
6
[153](#)
7
[153](#)
8
[153](#)
9
[154](#)
0
[154](#)
1
[154](#)
2
[154](#)
3
[154](#)
4
[154](#)
5
[154](#)
6
[154](#)
7
[154](#)
8
[154](#)
9
[155](#)
0
[155](#)
1
[155](#)
2
[155](#)
3
[155](#)
4
[155](#)
5
[155](#)
6
[155](#)
7
[155](#)
8
[155](#)
9
[156](#)
0
[156](#)
1
[156](#)
2
[156](#)
3
[156](#)

4
156
5
156
6
156
7
156
8
156
9
157
0
157
1
157
2
157
3
157
4
157
5
157
6
157
7
157
8
157
9
158
0
158
1
158
2
158
3
158
4
158
5
158
6
158
7
158
8
158
9
159
0
159
1
159
2
159
3
159
4
159
5
159
6
159
7
159
8
159
9
160
0
160
1
160
2
160
3

160
4
160
5
160
6
160
7
160
8
160
9
161
0
161
1
161
2
161
3
161
4
161
5
161
6
161
7
161
8
161
9
162
0
162
1
162
2
162
3
162
4
162
5
162
6
162
7
162
8
162
9
163
0
163
1
163
2
163
3
163
4
163
5
163
6
163
7
163
8
163
9
164
0
164
1
164
2
164

3
164
4
164
5
164
6
164
7
164
8
164
9
165
0
165
1
165
2
165
3
165
4
165
5
165
6
165
7
165
8
165
9
166
0
166
1
166
2
166
3
166
4
166
5
166
6
166
7
166
8
166
9
167
0
167
1
167
2
167
3
167
4
167
5
167
6
167
7
167
8
167
9
168
0
168
1
168
2

168
3
168
4
168
5
168
6
168
7
168
8
168
9
169
0
169
1
169
2
169
3
169
4
169
5
169
6
169
7
169
8
169
9
170
0
170
1
170
2
170
3
170
4
170
5
170
6
170
7
170
8
170
9
171
0
171
1
171
2
171
3
171
4
171
5
171
6
171
7
171
8
171
9
172
0
172
1
172

2
172
3
172
4
172
5
172
6
172
7
172
8
172
9
173
0
173
1
173
2
173
3
173
4
173
5
173
6
173
7
173
8
173
9
174
0
174
1
174
2
174
3
174
4
174
5
174
6
174
7
174
8
174
9
175
0
175
1
175
2
175
3
175
4
175
5
175
6
175
7
175
8
175
9
176
0
176
1

[176](#)
[2](#)
[176](#)
[3](#)
[176](#)
[4](#)
[176](#)
[5](#)
[176](#)
[6](#)
[176](#)
[7](#)
[176](#)
[8](#)
[176](#)
[9](#)
[177](#)
[0](#)
[177](#)
[1](#)
[177](#)
[2](#)
[177](#)
[3](#)
[177](#)
[4](#)
[177](#)
[5](#)
[177](#)
[6](#)
[177](#)
[7](#)
[177](#)
[8](#)
[177](#)
[9](#)
[178](#)
[0](#)
[178](#)
[1](#)
[178](#)
[2](#)
[178](#)
[3](#)
[178](#)
[4](#)
[178](#)
[5](#)
[178](#)
[6](#)
[178](#)
[7](#)
[178](#)
[8](#)
[178](#)
[9](#)
[179](#)
[0](#)
[179](#)
[1](#)
[179](#)
[2](#)
[179](#)
[3](#)
[179](#)
[4](#)
[179](#)
[5](#)
[179](#)
[6](#)
[179](#)
[7](#)
[179](#)
[8](#)
[179](#)
[9](#)
[180](#)
[0](#)
[180](#)

1
180
2
180
3
180
4
180
5
180
6
180
7
180
8
180
9
181
0
181
1
181
2
181
3
181
4
181
5
181
6
181
7
181
8
181
9
182
0
182
1
182
2
182
3
182
4
182
5
182
6
182
7
182
8
182
9
183
0
183
1
183
2
183
3
183
4
183
5
183
6
183
7
183
8
183
9
184
0

184
1
184
2
184
3
184
4
184
5
184
6
184
7
184
8
184
9
185
0
185
1
185
2
185
3
185
4
185
5
185
6
185
7
185
8
185
9
186
0
186
1
186
2
186
3
186
4
186
5
186
6
186
7
186
8
186
9
187
0
187
1
187
2
187
3
187
4
187
5
187
6
187
7
187
8
187
9
188

0
188
1
188
2
188
3
188
4
188
5
188
6
188
7
188
8
188
9
189
0
189
1
189
2
189
3
189
4
189
5
189
6
189
7
189
8
189
9
190
0
190
1
190
2
190
3
190
4
190
5
190
6
190
7
190
8